Machine Learning e Pattern Recognition

Luca & Mellow

5 maggio 2026

Indice

1 Introduction **2**

2 Dimensionality Reduction **5**
2.1 Principal Component Analysis (PCA) 5
2.2 Linear Discriminant Analysis (LDA) 8

3 Probability and Density Estimation **13**

4 Generative Gaussian Models **15**
4.1 Naive Bayes 19
4.2 Tied covariance matrix 21

5 Generative Multinomial Models **22**

6 Bayes Decisions and Model Evaluation **29**

7 Logistic Regression **41**

8 Gaussian Mixture Models **55**

9 Score Calibration and Fusion **70**

1 Introduction

Machine Learning: a computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

Pattern Recognition: automatic discovery of regularities in data through the use of computer algorithms to take actions such as classifying the data into different categories.

Supervised learning task: along with the input data the system is provided with output data. The goal is estimating a mapping between input and output data. Ex: classification.

Unsupervised learning task: no feedback is provided to the system, whose goal is to identify some useful structure of the data. Unsupervised methods are often used as pre-processing steps for supervised methods. Ex: density estimation

Underfitting: a too-simple model fits the observed data very poorly, and it is not able to provide accurate predictions for unseen data.

Overfitting: an over-complex model fits very accurately the observed data, but it is not able to provide accurate predictions for unseen data.

Curse of dimensionality: as the number of dimensions increases, the volume of the space grows exponentially, making data very sparse.

A **decision function** maps an input sample to a score or a label, determining the predicted class or outcome. It should provide a good generalization error.

The **generalization error of a decision function** is the expected difference between the predicted outputs and the true outputs on new, unseen data.

A **discriminative probabilistic model** directly estimates class posterior probabilities $P(C_k|\mathbf{x})$, without modeling how the data are generated. It focuses only on the decision boundary between classes. It does not describe the data distribution.

A **generative probabilistic model** estimates the class joint probability distribution $P(\mathbf{x}, C_k) = P(\mathbf{x}|C_k)P(C_k)$ (or equivalently the class likelihoods, and the class prior probabilities), and then applies Bayes' theorem to compute class posterior probabilities. It models how data is generated and can also

generate new samples. It is most demanding and informative.

[INTEGRAZIONE TEORICA]
La formula di Bayes usata dal modello generativo è:

$$P(C_k|\mathbf{x}) = \frac{P(\mathbf{x}|C_k)\,P(C_k)}{P(\mathbf{x})}$$

dove $P(\mathbf{x}|C_k)$ è la class-conditional likelihood, $P(C_k)$ è la prior, e $P(\mathbf{x})$ è l'evidenza (costante di normalizzazione).

A **model** is a mathematical or computational representation of the relationship between input (features) and output (labels) variables.

A **parametric model** depends on a set of parameters θ whose size does not depend on the available data.

A **training set** $\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)\}$

An **objective function** is a mathematical function that we want to minimize or maximize to solve a mathematical or optimization problem.

The **frequentist approach** is a statistical framework where probabilities are interpreted as long-run frequencies of events, and model parameters are considered fixed but unknown quantities. Since θ is unknown, we compute an estimate θ^* of its value from the training set. Usually, θ^* is obtained through the optimization of a suitable objective function. θ^* encodes all the information extracted from the dataset $\mathcal{D}$.

[INTEGRAZIONE TEORICA]
Nel formalismo frequentista, l'inferenza avviene approssimando i parametri stimati ai parametri veri:

- Modello generativo: $P(\mathbf{x}_t|C_k, \mathcal{D}, M) \approx P(\mathbf{x}_t|C_k, \theta^*, M)$
- Modello discriminativo: $P(C_k|\mathbf{x}_t, \mathcal{D}, M) \approx P(C_k|\mathbf{x}_t, \theta^*, M)$

The **Bayesian approach** is a statistical framework where probabilities represent degrees of belief, and model parameters are treated as random variables with their own probability distributions, which are updated as new data becomes available.

Define a prior distribution for θ. The prior distribution encodes our knowledge on the problem, before we actually see the data. The learning stage is

done by updating the prior distribution to take into account the training set data:

$$P(\theta|M),\ \mathcal{D} \longrightarrow P(\theta|\mathcal{D}, M)$$

The **posterior distribution density** $P(\theta|\mathcal{D}, M)$ encodes our knowledge on the model parameters, after we have seen the data. The inference stage is done by marginalizing (ie to integrate out) over the model parameters.

[INTEGRAZIONE TEORICA]
La formula di marginalizzazione per l'inferenza bayesiana (caso generativo) è:

$$P(\mathbf{x}_t|C_k, \mathcal{D}, M) = \int P(\mathbf{x}_t, \theta|C_k, M)\, P(\theta|\mathcal{D}, M)\, d\theta$$

The **Bayesian** approach models the uncertainty over the model parameter estimates.

The **Bayesian** approach is effective when training data is scarce, but both training and inference are significantly complex.

We assess the quality of our predictions. Since we cannot know the system performance on unseen data, we simulate its behavior on known data using an evaluation or test set.

Note that with time this may lead to biased conclusions, it is unavoidable for practical reasons.

Models often contain hyperparameters.

Hyperparameters are configuration variables set before the learning process begins, which control the behavior or structure of a machine learning algorithm but are not learned from the data.

We need a way to perform model selection. However, we cannot use test data to select good hyperparameters values, as this would bias our evaluation. We make use of a validation set. The validation set is used to assess the influence of model hyperparameters on prediction accuracy in order to select good hyperparameters values. So, part of the training set will be used for estimating our models. The remaining part, the validation set, will simulate the evaluation data, and will be used to infer hyperparameters and for model selection.

2 Dimensionality Reduction

Dimensionality reduction techniques compute a mapping from the n-dimensional feature space to a m-dimensional space, with $m \ll n$.

2.1 Principal Component Analysis (PCA)

PCA is a linear unsupervised dimensionality reduction technique. It works by finding a new set of orthogonal directions, called principal components, along which to project the data, in order to minimize the average reconstruction error.

Mathematically, PCA computes the eigenvectors and eigenvalues of the data covariance matrix: the first eigenvectors (with the largest eigenvalues) represent the main directions.

We are given a zero-mean dataset $\mathcal{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_K\}$, with $\mathbf{x}_i \in \mathbb{R}^n$.

We want to find the subspace of $\mathbb{R}^n$ that allows preserving most of the information.

A subspace can be represented as a matrix $P \in \mathbb{R}^{n \times m}$ whose columns are orthonormal.

The set of columns of P form a basis of a subspace of $\mathbb{R}^n$ with dimension m.

The projection of $\mathbf{x}$ over the subspace is given by $\mathbf{y} = P^T\mathbf{x}$.

We can compute the coordinates of the projected point $\mathbf{y}$ in the original space as $\hat{\mathbf{x}} = P\mathbf{y}$.

We need to define a criterion for estimating P. A reasonable criterion is the minimization of the average reconstruction error. We therefore want to solve:

$$P^* = \arg\min_P \frac{1}{K} \sum_{i=1}^{K} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 = \arg\min_P \frac{1}{K} \sum_{i=1}^{K} \|\mathbf{x}_i - PP^T\mathbf{x}_i\|^2$$

subject to $P^TP = I$.

Since Tr is a linear operator, minimizing $L(P)$ is equivalent to maximizing:

$$\widetilde{L}(P) = \mathrm{Tr}\left(P^T\left(\frac{1}{K}\sum_{i=1}^{K}\mathbf{x}_i\mathbf{x}_i^T\right)P\right)$$

We require that P is an orthogonal matrix.

The optimal solution is given by the matrix P whose columns are the m eigenvectors corresponding to the m largest eigenvalues of $\frac{1}{K}\sum_{i=1}^{K}\mathbf{x}_i\mathbf{x}_i^T$. Let:

$$\frac{1}{K}\sum_{i=1}^{K}\mathbf{x}_i\mathbf{x}_i^T = U\Sigma U^T$$

be an eigen-decomposition of the matrix, with Σ containing the eigenvalues in descending order

$$\Sigma = \begin{pmatrix}\sigma_1 & & & \\ & \sigma_2 & & \\ & & \ddots & \\ & & & \sigma_n\end{pmatrix}, \quad \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_n$$

and

$$U = [\mathbf{u}_1 \ldots \mathbf{u}_m, \mathbf{u}_{m+1} \ldots \mathbf{u}_n]$$

Then

$$P^* = [\mathbf{u}_1 \ldots \mathbf{u}_m]$$

As a consequence of projecting onto the eigenvectors of the covariance matrix, PCA transforms the data into a new coordinate system where the greatest variance lies on the first axis, the second greatest on the second, and so on. **This is why PCA can be interpreted as the linear mapping that preserves the directions with the highest variance.**

If the dataset is not zero-mean, the first PCA direction will approximately connect the origin and the dataset mean, which is generally not informative. It is therefore essential to center the data by subtracting the dataset mean $\bar{\mathbf{x}}$ before computing PCA. The PCA subspace P is then computed from the eigenvectors of the empirical covariance matrix:

$$C = \frac{1}{K} \sum_{i=1}^{K} (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T$$

which in 2D can be visualized as an ellipse whose axes correspond to the principal directions and whose lengths are proportional to the standard deviations.

Selection of optimal m, that is a hyperparameter, can be done by cross-validation or by selecting it to retain a given percentage t (e.g., 95%) of the variance of the data. We know that each eigenvalue corresponds to the variance along the corresponding axis. We choose m as:

$$\min_m \; m \quad \text{s.t.} \quad \frac{\sum_{i=1}^{m} \sigma_i}{\sum_{i=1}^{n} \sigma_i} \geq t$$

where σ_i is the i-th largest eigenvalue.

Pro and cons:

- Computing the sample covariance matrix can be difficult when the feature space is very large.
- Since PCA is unsupervised, we have no guarantee to obtain discriminant directions.
- Practical purposes of PCA:
  - Compress information
  - Remove unwanted variability (noise)
  - Simplify classification by reducing the number of parameters to estimate
  - Visualize the data
  - A further practical advantage is that PCA automatically eliminates directions with zero or very low variance, which could cause numerical instability in some classifiers (e.g., division by zero variance).

2.2 Linear Discriminant Analysis (LDA)

LDA is a linear supervised dimensionality reduction technique and a classification technique that finds linear combinations of features which best separate two or more classes.

We represent a direction as a unit vector $\mathbf{w}$.

The projected point is $y = \mathbf{w}^T\mathbf{x}$ (a scalar).

LDA projects the data into a lower-dimensional space by finding a direction that has a large separation between the classes and small spread inside each class. We measure spread in terms of class (co)variance. LDA maximizes the between-class variability s_B over within-class variability s_W ratio for the transformed samples:

$$\max_{\mathbf{w}} \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}$$

Since we are looking for a discriminant direction $\mathbf{w}$, we now consider the between s_B and within s_W class variance of the projected samples $\mathbf{w}^T\mathbf{x}$.

The global mean and class means in the direction $\mathbf{w}$ are simply:

$$m = \mathbf{w}^T\boldsymbol{\mu}, \quad m_c = \mathbf{w}^T\boldsymbol{\mu}_c$$

The between and within class variance over the direction $\mathbf{w}$ can also be computed as:

$$s_B = \frac{1}{N}\sum_{c=1}^{K} n_c(\mathbf{w}^T\boldsymbol{\mu}_c - \mathbf{w}^T\boldsymbol{\mu})(\mathbf{w}^T\boldsymbol{\mu}_c - \mathbf{w}^T\boldsymbol{\mu})^T = \mathbf{w}^T S_B \mathbf{w}$$

$$s_W = \frac{1}{N}\sum_{c=1}^{K}\sum_{i=1}^{n_c}(\mathbf{w}^T\mathbf{x}_{c,i} - \mathbf{w}^T\boldsymbol{\mu}_c)(\mathbf{w}^T\mathbf{x}_{c,i} - \mathbf{w}^T\boldsymbol{\mu}_c)^T = \mathbf{w}^T S_W \mathbf{w}$$

where the between and within class variability matrices are defined as:

$$S_B \triangleq \frac{1}{N}\sum_{c=1}^{K} n_c(\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^T$$

$$S_W \triangleq \frac{1}{N} \sum_{c=1}^{K} \sum_{i=1}^{n_c} (\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)(\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)^T$$

As a criterion of optimality we define the maximization of the ratio of s_B and s_W. We assume that S_W is full rank, thus the objective function is:

$$L(\mathbf{w}) = \frac{s_B}{s_W} = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}$$

Note that the criterion does not depend on the scale of $\mathbf{w}$, so we can select a maximizer with unit form.

We can find an optimum by solving:

$$\nabla_{\mathbf{w}} L(\mathbf{w}) = \frac{2S_B\mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}} - \frac{2\mathbf{w}^T S_B \mathbf{w} \cdot S_W \mathbf{w}}{(\mathbf{w}^T S_W \mathbf{w})^2} = 0$$

The optimum is obtained for:

$$(\mathbf{w}^T S_W \mathbf{w}) S_B \mathbf{w} = (\mathbf{w}^T S_B \mathbf{w}) S_W \mathbf{w}$$

i.e.

$$S_W^{-1} S_B \mathbf{w} = \lambda(\mathbf{w}) \mathbf{w}$$

where

$$\lambda(\mathbf{w}) = L(\mathbf{w}) = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}$$

The maximum of L is thus the eigenvector corresponding to the largest eigenvalue of $S_W^{-1} S_B$.

[INTEGRAZIONE TEORICA]
dalla derivata poni $\nabla L = 0$ e ottieni (dopo la regola del quoziente): dividi entrambi i lati per lo scalare $(\mathbf{w}^T S_W \mathbf{w})$

- **Passo 1**
  Dalla derivata poni $\nabla L = 0$ e ottieni (dopo la regola del quozien-

te):
$(\mathbf{w}^T S_W \mathbf{w}) \cdot S_B \mathbf{w} = (\mathbf{w}^T S_B \mathbf{w}) \cdot S_W \mathbf{w}$
I due termini tra parentesi $(\mathbf{w}^T S_W \mathbf{w})$ e $(\mathbf{w}^T S_B \mathbf{w})$ sono due scalari — sono numeri, non vettori. Il risultato della moltiplicazione vettore trasposto × matrice × vettore è sempre un numero.

- **Passo 2**
  Dividi entrambi i lati per lo scalare $(\mathbf{w}^T S_W \mathbf{w})$
  $S_B \mathbf{w} = \left[\frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}\right] \cdot S_W \mathbf{w}$

- **Passo 3**
  Moltiplica entrambi i lati per S_W^{-1}
  $S_W^{-1} S_B \mathbf{w} = \left[\frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}\right] \cdot \mathbf{w}$

- **Passo 4**
  Riconosci il termine tra parentesi. Quel rapporto tra parentesi è esattamente $L(\mathbf{w})$, la funzione obiettivo definita all'inizio! Quindi lo chiami $\lambda(\mathbf{w})$ per brevità, e ottieni:
  $S_W^{-1} S_B \mathbf{w} = \lambda(\mathbf{w}) \cdot \mathbf{w}$

Perché questo è un problema agli autovalori?
Perché ha esattamente la forma $M\mathbf{v} = \lambda\mathbf{v}$ — una matrice moltiplicata per un vettore dà lo stesso vettore scalato. Qui $M = S_W^{-1} S_B$, $\mathbf{v} = \mathbf{w}$, $\lambda = L(\mathbf{w})$.

La conclusione chiave
Poiché $\lambda(\mathbf{w}) = L(\mathbf{w})$, massimizzare L significa massimizzare λ. E tra tutti gli autovettori di $S_W^{-1} S_B$, quello che massimizza λ è per definizione quello con autovalore più grande. Quindi la $\mathbf{w}$ ottimale è quell'autovettore.

For a binary problem, we can express:

$$S_B = k(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T$$

where k is a constant. An eigenvector associated to the (single) non-zero eigenvalue is:

$$\mathbf{w} \propto S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$$

To derive a classification rule, we consider a classifier based on the distance of a sample from the class means, in the projected space.

Let $m_1 = \mathbf{w}^T\boldsymbol{\mu}_1$, $m_2 = \mathbf{w}^T\boldsymbol{\mu}_2$. The sample $\mathbf{x}_t$ is assigned label C_1 if:

$$|\mathbf{w}^T\mathbf{x}_t - m_1|^2 < |\mathbf{w}^T\mathbf{x}_t - m_2|^2$$

Assuming we chose $\mathbf{w}$ so that $m_2 - m_1 > 0$, our assignment becomes:

$$C(\mathbf{x}_t) = \begin{cases} C_1 & \text{if } \mathbf{w}^T\mathbf{x}_t < t \\ C_2 & \text{if } \mathbf{w}^T\mathbf{x}_t \geq t \end{cases}$$

where $t = \frac{m_1+m_2}{2}$.

Despite being originally introduced to solve binary classification problems, LDA has found a large success as a dimensionality reduction technique. In this case, **we are interested in looking for the m most discriminant directions. We represent these directions as a matrix $W \in \mathbb{R}^{n\times m}$, whose columns contain the directions we want to find.** We do not require that W is orthogonal. We have:

$$\widetilde{\mathbf{x}} = W^T\mathbf{x}$$

We can express the transformed between and within class covariance matrices as:

$$\widetilde{S}_B = W^T S_B W$$

$$\widetilde{S}_W = W^T S_W W$$

A criteria is to look for the maximizer of:

$$L = \mathrm{Tr}\left(\tilde{S}_W^{-1}\tilde{S}_B\right)$$

Here, the solution is given by the m eigenvectors corresponding to the m largest eigenvalues of $S_W^{-1} S_B$.

Approfondimento: metodo alternativo

Another criteria is to solve the generalized eigenvalue problem $S_B \mathbf{w} = \lambda S_W \mathbf{w}$. It consists in the joint diagonalization of S_W and S_B that makes S_W become the identity matrix and S_B become a diagonal matrix:
Whiten S_W:
Compute the eigenvalue decomposition $S_W = U_W \Sigma_W U_W^T$.
Apply the whitening transformation described by:

$$P_W = U_W \Sigma_W^{-\frac{1}{2}} U_W^T$$

This transforms:

$$S_W \to I, \quad S_B \to P_W S_B P_W^T$$

Diagonalize the transformed S_B:
Compute the eigenvalue decomposition $P_W S_B P_W^T = U_B \Sigma_B U_B^T$.
Diagonalize by projecting over U_B^T:

$$S_W \to I, \quad S_B \to \Sigma_B$$

The first m directions of the transformed samples correspond to the LDA subspace.

Pro and cons:

- LDA assumes that the variability of data within each class follows a Gaussian distribution.
- When the number of directions is large compared to the number of samples, S_W can become singular or close to singular, making it difficult or impossible to compute its inverse.
- It is often useful to pre-process data using PCA before applying LDA.
- Notice that, from the definition of S_B, the number of non-zero eigenvalues is at most $K - 1$ (where K is the number of classes), therefore LDA allows estimating at most $K - 1$ directions.

3 Probability and Density Estimation

$P : \mathcal{A} \longrightarrow \mathbb{R}^+, \quad P(\Omega) = 1$

$P(\bigcup_{n=1}^{\infty} A_n) = \sum_{n=1}^{\infty} P(A_n)$ (countable additivity, eventi mutuamente esclusivi)

$P(\emptyset) = 0, \quad P(A^C) = 1 - P(A), \quad P(A \cup B) = P(A) + P(B) - P(A \cap B)$

$P(A|B) = \frac{P(A \cap B)}{P(B)}$

$P(B|A) = \frac{P(A|B)\,P(B)}{P(A)}$ (Bayes formula)

$P(A \cap B) = P(A)\,P(B) \iff A \perp\!\!\!\perp B$ (indipendenza)

$X : \Omega \longrightarrow \mathbb{R}, \quad P(X \leq x) = P(\{\omega \in \Omega : X(\omega) \leq x\})$

$F_X(x) = P(X \leq x)$

$f_X(x) \geq 0, \quad \int_{-\infty}^{+\infty} f_X(x)\,dx = 1$

$f_X(x) = \frac{d}{dx} F_X(x)$

$P(a \leq X \leq b) = \int_a^b f_X(x)\,dx$

$f_{\mathbf{X}}(\mathbf{x}) = \frac{\partial^m}{\partial x_1 \cdots \partial x_m} F_{\mathbf{X}}(\mathbf{x}), \quad f_{X_1}(x_1) = \int_{\mathbb{R}^{m-1}} f_{\mathbf{X}}(x_1, y_2, \ldots, y_m)\,dy_2 \cdots dy_m$

$f_{X|Y}(x|y) = \frac{f_{X,Y}(x,y)}{f_Y(y)}, \quad f_{Y|X}(y|x) = \frac{f_{X|Y}(x|y)\,f_Y(y)}{f_X(x)}$

$E_X[X] = \int_S x\,f_X(x)\,dx$

$\mathrm{var}(X) = E_X[(X - E_X[X])^2] = E_X[X^2] - E_X[X]^2$

$E_X\left[X^k\right] = \int_S x^k f_X(x)\,dx$ (k-th moment), $\qquad E_X\left[(X - E_X[X])^k\right]$ (central k-th moment)

$E_X[g(X)] = \int_S g(x)\,f_X(x)\,dx$

$\mathrm{cov}(X, Y) = E_{X,Y}[XY] - E_X[X]\,E_Y[Y]$

$\Sigma = \mathrm{cov}(\mathbf{X}) = E\left[(\mathbf{X} - E[\mathbf{X}])(\mathbf{X} - E[\mathbf{X}])^T\right] = E\left[\mathbf{X}\mathbf{X}^T\right] - E[\mathbf{X}]\,E[\mathbf{X}]^T$

$X \sim \mathrm{Ber}(p) : \quad P_X(x) = \mathrm{Ber}(x|p) = p^x(1-p)^{1-x}$

$E[X] = p, \quad \mathrm{var}(X) = p(1-p)$

$X \sim \mathrm{Bin}(n, p) : \quad P_X(x) = \binom{n}{x} p^x (1-p)^{n-x}$

$X_1 \perp\!\!\!\perp \cdots \perp\!\!\!\perp X_n \sim \mathrm{Ber}(p) \implies Y = \sum_i X_i \sim \mathrm{Bin}(n, p)$

$E[X] = np, \quad \text{var}(X) = np(1-p)$

$$\mathbb{1}[C] = \begin{cases} 1 & \text{if } C \text{ is true} \\ 0 & \text{otherwise} \end{cases}$$

$X \sim \text{Cat}(\mathbf{p}): \quad f_X(x) = \prod_i p_i^{\mathbb{1}[x=i]}, \qquad \mathbf{p} = (p_1, \ldots, p_K), \quad \sum_{i=1}^K p_i = 1$

$X \sim \text{Mul}(n, \mathbf{p}): \quad f_X(\mathbf{x}) = \frac{n!}{x_1! \cdots x_K!} \prod_{i=1}^K p_i^{x_i}$

$X_1 \perp\!\!\!\perp \cdots \perp\!\!\!\perp X_n \sim \text{Cat}(\mathbf{p}) \implies Y = \sum_{i=1}^n X_i \sim \text{Mul}(n, \mathbf{p})$

$X \sim \mathcal{N}(\mu, \sigma^2): \quad f_X(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$

$E[X] = \mu, \quad \text{var}(X) = \sigma^2, \quad \lambda = \frac{1}{\sigma^2}$ (precision)

$\mathbf{X} \sim \mathcal{N}(\mathbf{0}, I): \quad f_{\mathbf{X}}(\mathbf{x}) = (2\pi)^{-N/2} e^{-\frac{1}{2}\mathbf{x}^T\mathbf{x}}$

$\mathbf{X} = A\mathbf{Y} + \boldsymbol{\mu}, \quad \mathbf{Y} \sim \mathcal{N}(\mathbf{0}, I), \quad \Sigma = AA^T \implies \mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$

$\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma): \quad f_{\mathbf{X}}(\mathbf{x}) = (2\pi)^{-N/2} |\Sigma|^{-1/2} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x}-\boldsymbol{\mu})\right)$

$\Lambda = \Sigma^{-1}$ (precision matrix)

$\frac{S_N - N\mu}{\sigma\sqrt{N}} \xrightarrow{d} \mathcal{N}(0,1), \quad S_N = \sum_{i=1}^N X_i$

$\lim_{n\to\infty} F_{S_N}(x) = F_{\mathcal{N}(0,1)}(x)$

Central Limit Theorem: given a sufficiently large sample size, the sum of i.i.d. (independent and identically distributed) random variables, each with finite mean and variance, will be approximately normally distributed, regardless of the original distribution of the variables.

4 Generative Gaussian Models

A **generative Gaussian model** is a generative probabilistic model that assumes the data for each class follow a multivariate Gaussian distribution, with its own mean vector and covariance matrix. This model estimates the parameters of the multivariate Gaussian distributions for each class and uses Bayes' theorem to compute the posterior probability that a new sample belongs to each class. According to the optimal Bayes decision rule, the model assigns the sample to the class with the highest probability. This model is called generative because it explicitly models the data generation process for each class, not just the decision boundary.

Let's consider a closed set classification problem, where we want to assign a test point x_t to one of k possible classes. We model x_t as a realization of a continuous random variable X_t, and its class label as a random variable $C_t \in \{1 \dots k\}$. All samples are assumed to be jointly i.i.d..

Let the joint density of (X, C) be:

$$f_{X_t,C_t}(x_t, c) = f_{X,C}(x_t, c)$$

From Bayes' theorem we can compute the class posterior probability:

$$P(C_t = c \mid X_t = x_t) = \frac{f_{X,C}(x_t, c)}{\sum_{c' \in C} f_{X,C}(x_t, c')}$$

We factorized the joint density in class likelihood and class prior:

$$f_{X,C}(x_t, c) = f_{X|C}(x_t \mid c) \cdot P_C(c)$$

The class likelihood $f_{X|C}(x \mid c, \theta)$ describes the distribution of the data within each class and depends on the model parameters.

The class prior $P_C(c)$, on the other hand, reflects the probability of a sample belonging to a class before seeing the data and is determined by the application context, not by the model parameters.

It's important to distinguish between the application prior, which represents the true class probabilities in the real world, and the empirical prior, which is simply the class frequency observed in the dataset. If the application prior is unknown, we often estimate it using the empirical prior, but these may

not always match, especially if the dataset is not representative of the real scenario.

Let's move on.

To proceed, we assume that the class likelihood for each class can be modeled by a multivariate Gaussian distribution:

$$(X_t \mid C_t = c) \sim \mathcal{N}(\mu_c,\, \Sigma_c)$$

We have one mean and one covariance matrix for each class. If we knew μ_c, Σ_c, then we could compute the class likelihood as:

$$f_{X_t|C_t}(x_t \mid c) = \mathcal{N}(x_t \mid \mu_c,\, \Sigma_c)$$

We do not know the values for the model parameters $\theta = [(\mu_1, \Sigma_1) \dots (\mu_k, \Sigma_k)]$, but, using the labeled training dataset $\mathcal{D} = \{(x_1, c_1) \dots (x_n, c_n)\}$, we can learn them.

We assume that, given the model parameters θ, observations are i.i.d..

We use ML estimation to compute the parameters.

The data likelihood factorizes as:

$$L(\theta) = \prod_{i=1}^{n} f_{X,C|\theta}(x_i, c_i \mid \theta) = \prod_{i=1}^{n} \mathcal{N}(x_i \mid \mu_{c_i}, \Sigma_{c_i})\, P(c_i)$$

Considering the log-likelihood:

$$\ell(\theta) = \sum_{c=1}^{k} \ell_c(\mu_c, \Sigma_c) + \xi \qquad \ell_c(\mu_c, \Sigma_c) = \sum_{i|c_i=c} \log \mathcal{N}(x_i \mid \mu_c, \Sigma_c)$$

The log-density for a Gaussian distribution $\mathcal{N}(\mu, \Sigma)$ is:

$$\log \mathcal{N}(x \mid \mu, \Sigma) = -\frac{D}{2} \log 2\pi - \frac{1}{2} \log |\Sigma| - \frac{1}{2}(x - \mu)^T \Sigma^{-1} (x - \mu)$$

The log-likelihood $\ell_c(\mu_c, \Sigma_c)$ can be rewritten in different ways:

$$\ell_c(\mu_c, \Sigma_c) = \kappa + \frac{N_c}{2} \log |\Lambda_c| - \frac{1}{2} \operatorname{Tr} \left(\Lambda_c \sum_{i|c_i=c} (x_i - \mu_c)(x_i - \mu_c)^T \right)$$

$$= \kappa + \frac{N_c}{2} \log |\Lambda_c| - \frac{1}{2} \operatorname{Tr} \left(\Lambda_c \sum_{i|c_i=c} x_i x_i^T \right) + \mu_c^T \Lambda_c \sum_{i|c_i=c} x_i - \frac{N_c}{2} \mu_c^T \Lambda_c \mu_c$$

The log-likelihood depends on the data only through the sufficient statistics:

$$Z_c = N_c, \qquad F_c = \sum_{i|c_i=c} x_i, \qquad S_c = \sum_{i|c_i=c} x_i x_i^T$$

We can find the ML solution by setting the gradients to zero. From (2), the derivative w.r.t. μ_c gives:

$$\mu_c^* = \frac{1}{N_c} \sum_{i|c_i=c} x_i$$

From (1), the derivative w.r.t. Λ_c gives:

$$\Sigma_c^* = \Lambda_c^{-1} = \frac{1}{N_c} \sum_{i|c_i=c} (x_i - \mu_c^*)(x_i - \mu_c^*)^T$$

We can now use the estimated parameters to compute the class likelihood:

$$f_{X_t|C_t}(x_t \mid c) \approx \mathcal{N}(x_t \mid \mu_c^*, \Sigma_c^*)$$

Let's consider a binary task, with 2 classes $C \in \{h_1, h_0\}$. We assign the label to a test sample x_t according to the highest posterior probability, comparing $P(C = h_1 \mid x_t)$ to $P(C = h_0 \mid x_t)$. We can express the comparison in terms of class posterior ratio:

$$\log r(x_t) = \log \frac{P(C = h_1 \mid x_t)}{P(C = h_0 \mid x_t)} = \log \frac{f_{X|C}(x_t \mid h_1)}{f_{X|C}(x_t \mid h_0)} + \log \frac{\pi}{1 - \pi}$$

The first term is the log-likelihood ratio (LLR) and represents the ratio between the likelihood of observing the sample given that it belongs to h_1 or to h_0:

$$\text{llr}(x_t) = \log \frac{f_{X|C}(x_t \mid h_1)}{f_{X|C}(x_t \mid h_0)}$$

The second term is the log-odds that is the logarithm of the ratio between the probability of an event and the probability of its complement.

Our system should be designed to output the LLR, so that it remains as independent as possible from the specific application.

The optimal decision is based on the comparison:

$$\log r(x_t) \gtrless 0$$

which means:

$$\text{llr}(x_t) = \log \frac{f_{X|C}(x_t \mid h_1)}{f_{X|C}(x_t \mid h_0)} \gtrless -\log \frac{\pi}{1-\pi}$$

The LLR acts as a score, with a probabilistic interpretation. Greater scores values imply our system favors class h_1, lower values mean it favors class h_0. The decision requires comparing the LLR to a threshold t that depends on the application, through the class prior probability π.

We can compute the LLR and obtaining the decision function:

$$\text{llr}(x) = x^T A x + x^T b + c$$

with:

$$A = -\frac{1}{2}(\Lambda_1 - \Lambda_0)$$

$$b = \Lambda_1 \mu_1 - \Lambda_0 \mu_0$$

$$c = -\frac{1}{2}\left(\mu_1^T \Lambda_1 \mu_1 - \mu_0^T \Lambda_0 \mu_0\right) + \frac{1}{2}(\log|\Lambda_1| - \log|\Lambda_0|)$$

A decision function assigns a class label to each point in the feature space. The decision surfaces are the boundaries in the feature space where the decision function changes value, that is, where the assigned class switches from one to another.

Let's consider a multiclass task $C \in \{h_1, h_2 \dots h_k\}$.

We can compute posterior probabilities:

$$P(C = h_i \mid x_t) = \frac{f_{X|C}(x_t \mid h_i)\, P(h_i)}{\sum_{h' \in \{h_1,\dots,h_k\}} f_{X|C}(x_t \mid h')\, P(h')}$$

Optimal decisions require choosing the class with highest posterior probability $c_t^* = \arg\max_h P(C = h \mid x_t)$. Posterior probabilities are proportional to $f_{X|C}(x_t \mid h)\, P(h)$, and the proportionality factor is the same for all classes.

Optimal decision can thus be computed as:

$$c_t^* = \arg\max_h \; f_{X|C}(x_t \mid h)\, P(h) = \arg\max_h \; \log f_{X|C}(x_t \mid h) + \log P(h)$$

Again, the first term should be the output of the classifier, whereas the second term depends on the application.

The Multivariate Gaussian model requires computing a mean and a covariance matrix for each class. It performs better if we have enough data to reliably estimate the covariance matrices. If the samples are few compared to their dimensionality, then the estimates can be inaccurate. The issue is more evident for covariance matrices, since they have $\frac{D(D+1)}{2}$ independent elements.

4.1 Naive Bayes

Naive Bayes is a model that assumes feature independence given the class. In other words, if, for each class, the different features (components) are approximately independent, we can assume that the class likelihood can be factorized over its components:

$$f_{X|C}(x \mid c) \approx \prod_{j=1}^{D} f_{X^{[j]}|C}(x^{[j]} \mid c)$$

In practice, this means that the class likelihood is the product of the density of each feature.

Note that the assumption is not tied to any specific distribution: we can even employ a different distribution family for each component.

Naive Bayes can simplify the estimation, but may perform poorly if data are high correlated.

In the Naive Bayes Gaussian model, we assume that features are independent given the class, and that each feature follows a normal distribution within each class. The class likelihood is computed as the product of the normal densities for each feature, each with its own mean and variance for that class:

$$f_{X^{[j]}|C}(x^{[j]} \mid c) = \mathcal{N}(x^{[j]} \mid \mu_{c,[j]}, \sigma^2_{c,[j]})$$

We can compute the ML estimates. We can optimize the log-likelihood independently for each component. For each component we have the log-likelihood of a normal model. So, the ML solution is:

$$\mu^*_{c,[j]} = \frac{1}{N_c} \sum_{i|c_i=c} x_{i,[j]}, \qquad \sigma^2_{c,[j]} = \frac{1}{N_c} \sum_{i|c_i=c} (x_{i,[j]} - \mu_{c,[j]})^2$$

We can observe that the density for a sample x can be expressed as:

$$f_{X|C}(x \mid c) = \prod_{j=1}^{D} \mathcal{N}(x^{[j]} \mid \mu^*_{c,[j]}, \sigma^2_{c,[j]}) = \mathcal{N}(x \mid \mu_c, \Sigma_c)$$

where:

$$\mu_c = \begin{pmatrix} \mu_{c,} \\ \mu_{c,} \\ \vdots \\ \mu_{c,[D]} \end{pmatrix}, \qquad \Sigma_c = \begin{pmatrix} \sigma^2_{c,} & 0 & \cdots & 0 \\ 0 & \sigma^2_{c,} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma^2_{c,[D]} \end{pmatrix}$$

The Naive Bayes Gaussian classifier corresponds to a Multivariate Gaussian classifier with diagonal covariance matrices (this does not hold in general).

4.2 Tied covariance matrix

The tied covariance matrix is the assumption that all classes share the same covariance matrix. A pro is that a single shared covariance matrix can be more easily estimated. Tied covariance models can capture correlations, but may perform poorly when classes have very different distributions. On the other hand, if we have reason to believe that covariances should be very similar, then the model will provide a more reliable estimate.

In the tied covariance matrix Gaussian model, each class is modeled with a multivariate Gaussian distribution, but all classes share the same covariance matrix. This means that while each class can have its own mean vector, the covariance matrix is the same for all classes:

$$f_{X|C}(x \mid c) = \mathcal{N}(x \mid \mu_c, \Sigma)$$

We can estimate the parameters using the ML framework. We get:

$$\mu_c^* = \frac{1}{N_c} \sum_{i|c_i=c} x_i, \qquad \Sigma^* = \frac{1}{N} \sum_c \sum_{i|c_i=c} (x_i - \mu_c)(x_i - \mu_c)^T$$

where $N = \sum_{c=1}^{k} N_c$ is the total number of samples.

Note that the binary LLR:

$$\text{llr}(x) = \log \frac{\mathcal{N}(x;\, \mu_1,\, \Lambda^{-1})}{\mathcal{N}(x;\, \mu_0,\, \Lambda^{-1})} = x^T b + c$$

with:

$$b = \Lambda(\mu_1 - \mu_0), \qquad c = -\frac{1}{2}\left(\mu_1^T \Lambda \mu_1 - \mu_0^T \Lambda \mu_0\right)$$

i.e., a **linear** function of x.

5 Generative Multinomial Models

Generative Multinomial Models

GENERATIVE MULTINOMIAL MODELS

The generative categorical model is a probabilistic model used for classification tasks where each sample is described by a single categorical feature (such as the color of an object).

The generative multinomial model is a probabilistic model used for classification tasks where each sample is represented by counts over a fixed set of categories (such as word counts in text classification).

In both models, for each class, the data are assumed to be generated according to a categorical or multinomial distribution, respectively, with class-specific parameters.

The model estimates the probability of each category within each class and uses these probabilities to compute the likelihood of a new sample belonging to each class.

Using Bayes' theorem, the model combines the class prior and the class likelihood (from the respective distribution) to compute the posterior probability for each class. The sample is then assigned to the class with the highest posterior probability.

The main difference is that the categorical model is suited for a single categorical feature (a single trial) or several categorical features that are modeled independently using the Naive Bayes assumption, while the multinomial model is designed for feature vectors representing counts over multiple categories (multiple trials). Both models are generative and use ML estimation for parameter learning.

We consider a classification problem modeled with a generative categorical approach. The following assumptions are made:

Features are discrete.

Each sample is described by a single categorical feature $x \in \{1 \dots m\}$, which can take one of m possible values.

We have a set of labeled training samples $\mathcal{D} = \{(x_1, c_1) \dots (x_n, c_n)\}$

All samples are i.i.d.

We want to compute the class likelihood for each class $c \in \{1 \dots k\}$:

$$L(\Pi) = \prod_{i=1}^{n} P(X_i = x_i \mid C_i = c_i) P(C_i = c_i)$$

The log-likelihood is given by:

$$\ell(\Pi) = \sum_{i=1}^{n} \log P(X_i = x_i \mid C_i = c_i) + \xi$$

$$= \sum_{i=1}^{n} \log \pi_{c_i, x_i} + \xi$$

$$= \sum_{c=1}^{k} \sum_{i:c_i=c} \log \pi_{c, x_i} + \xi$$

$$= \sum_{c=1}^{k} \ell_c(\pi_c) + \xi$$

where:

$$\ell_c(\pi_c) = \sum_{i:c_i=c} \log \pi_{c, x_i}$$

So, $\ell(\Pi)$ can be expressed as a sum of terms, each depending on a separate subset of the parameters, those of class c.

We can thus estimate the parameters by independently optimizing $\ell_c(\pi_c)$:

$$\ell_c(\pi_c) = \sum_{i:c_i=c} \log \pi_{c, x_i} = \sum_{j=1}^{m} N_{c,j} \log \pi_{c,j}$$

where $N_{c,j}$ is the number of times we observed $x_i = j$ in the dataset for class c.

Since we have the constraint $\sum_{j=1}^{m} \pi_{c,j} = 1$, we use the Lagrange multipliers and look for the stationary points of (note that we drop subscript c):

$$L(\pi) = f(\pi) - \lambda \left(\sum_{j=1}^{m} \pi_j - 1 \right) = \sum_{j=1}^{m} N_j \log \pi_j - \lambda \left(\sum_{j=1}^{m} \pi_j - 1 \right)$$

We need to solve:

$$\frac{\partial L}{\partial \pi_i} = \frac{N_i}{\pi_i} - \lambda = 0 \quad \forall\, i = 1 \dots m$$

$$\frac{\partial L}{\partial \lambda} = 1 - \sum_{j=1}^{m} \pi_j = 0$$

From the first equation: $\pi_i = \dfrac{N_i}{\lambda}$. Substituting into the second:

$$\sum_{j=1}^{m} \pi_j = \frac{1}{\lambda} \sum_{j=1}^{m} N_j = 1 \implies \lambda = \sum_{j=1}^{m} N_j = N$$

Therefore:

$$\pi_i = \frac{N_i}{N}$$

The ML solution for each class is thus:

$$\pi^*_{c,i} = \frac{N_{c,i}}{N_c}$$

and we can compute the class likelihood as:

$$P(X_t = x_t \mid C_t = c) = \pi^*_{c,x_t}$$

If we have multiple categorical features, we could model their joint probability as a single categorical random variable whose values correspond to all possible combinations of the features. However, the number of possible combinations grows exponentially and quickly becomes intractable.

To address this, we can adopt the Naive Bayes assumption, treating the features as independent. This allows us to estimate the parameters for each feature separately using ML estimation, and then combine them to model the overall likelihood:

$$P(X_t = x_t \mid C_t = c) = \prod_j \pi_{c,x_t,[j]}$$

where j denotes the feature index.

We consider a classification problem modeled with a generative multinomial approach. The following assumptions are made:

Features are discrete.

Each sample is represented by a feature vector $\mathbf{x} = (x \dots x[m])$, where each $x[j]$ indicates the count of occurrences of a specific event or category, and each feature vector is modeled using a multinomial distribution.

We have a set of labeled training samples $\mathcal{D} = \{(x_1, c_1) \dots (x_n, c_n)\}$

All samples are i.i.d.

Thus, for each class c, we have a set of parameters:

$$\pi_c = (\pi_{c,1} \dots \pi_{c,m})$$

The probability for feature vector $\mathbf{x}$ is given by the multinomial density:

$$P(X = x \mid C = c) = \frac{(\sum_{j=1}^m x[j])!}{\prod_{j=1}^m x[j]!} \prod_{j=1}^m \pi_{c,j}^{x[j]} \propto \prod_{j=1}^m \pi_{c,j}^{x[j]}$$

We can use the ML estimation. As in the previous case, the likelihood factorizes over classes, thus:

$$\ell(\Pi) = \sum_c \ell_c(\pi_c) + \xi$$

with:

$$\ell_c(\pi_c) = \sum_{i:c_i=c} \sum_{j=1}^{m} x_{i,[j]} \log \pi_{c,j}$$

Note that we did not write the multinomial coefficient since it is constant with respect to Π and thus can be absorbed in ξ.

Also in this case we can express the log-likelihood as:

$$\ell_c(\pi_c) = \sum_{j=1}^{m} N_{c,j} \log \pi_{c,j}$$

where:

$$N_{c,j} = \sum_{i:c_i=c} x_{i,[j]}$$

is the total number of occurrences of event j (word j) in the samples of class c.

The problem has exactly the same form as the one we solved for the categorical case. Thus the ML solution is again:

$$\pi_{c,j} = \frac{N_{c,j}}{N_c}$$

where N_c is the total number of words for class c:

$$N_c = \sum_j N_{c,j} = \sum_{i:c_i=c} \sum_{j=1}^{m} x_{i,[j]}$$

Let's consider a binary task for a multinomial case. We write the log-likelihood ratio:

$$llr(x) = \log \frac{P(X = x \mid C = h_1)}{P(X = x \mid C = h_0)} = \sum_{j=1}^{m} x[j] \log \pi_{h_1,j} - \sum_{j=1}^{m} x[j] \log \pi_{h_0,j} = x^T b$$

with $\mathbf{b} = (\log \pi_{h_1,1} - \log \pi_{h_0,1}, \ldots, \log \pi_{h_1,m} - \log \pi_{h_0,m})$.

Again, we have linear decision functions.

Rare feature values can cause problems: if a feature value does not appear in a class, we will estimate a probability of zero for that value in the class. Any test sample containing that feature value will then have zero probability of belonging to the class.

To address this, we can introduce pseudo-counts, assuming that each class contains at least one occurrence of every possible feature value. In practice, this means adding a fixed value to the observed counts before computing the ML solution.

This approach corresponds to a Maximum-a-posterior estimate using Dirichlet prior distribution.

Just some notes.

A Bayesian treatment of the hyperparameters is a more appropriate way to deal with limited sample sizes.

If we want to partially account for correlations between features, we can consider combinations of feature values (such as pairs or triplets) instead of modeling each feature independently.

Remember that we can also combine different models (categorical, multinomial, ...) through a Naive Bayes assumption.

Note that for both the categorical model and the multinomial model, the ML solution for the parameters $\pi_{c,j}$ corresponds to the relative frequencies of occurrences.

We can go further and show that the two models are equivalent. Considering only samples of a single class, in the first case the log-likelihood for a given class is given by:

$$\ell_X(\pi) = \sum_{i=1}^{n} \log \pi_{x_i} = \sum_{j=1}^{m} N_j \log \pi_j$$

whereas in the second case:

$$\ell_Y(\pi) = \log \frac{(\sum_{j=1}^{m} y[j])!}{\prod_{i=1}^{m} y[j]!} + \sum_{j=1}^{m} y_j \log \pi_j = \xi + \sum_{j=1}^{m} N_j \log \pi_j$$

The corresponding likelihoods $L_X(\pi)$ and $L_Y(\pi)$ are proportional:

$$L_X(\pi) \propto L_Y(\pi)$$

Therefore, the ML estimates will be the same in both cases. The Bayesian posterior probabilities for the model parameters will also be identical. As a result, inference does not change: for any given test sample, the class likelihoods remain proportional, leading to equal binary log-likelihood ratios and identical class posterior probabilities.

6 Bayes Decisions and Model Evaluation

Bayes Decisions and Model Evaluation

Maximum-a-posteriori class assignment is an approach that does not consider the fact that different labeling errors may have a different impact. Thus, in some contexts, it would not necessarily lead to the best decision.

A **metric** is a quantitative measure used to evaluate the performance of a model or algorithm. In machine learning, metrics help assess how well a model makes predictions. Two simple metrics are **accuracy** and **error rate**:

accuracy:

$$\text{accuracy} = \frac{\#\text{ of correctly classified samples}}{\#\text{ of samples}}$$

error rate:

$$\text{error rate} = \frac{\#\text{ of incorrectly classified samples}}{\#\text{ of samples}} = 1 - \text{accuracy}$$

However, although in some scenarios accuracy can be a good metric, it's affected by several issues that may result in the metric being less useful for the evaluation of a classifier, or the comparison of different classifiers. These issues are:

- it does not account for the cost of the different kinds of errors
- it is dependent on the empirical class prior of the different classes, which does not necessarily reflect the application prior
- it is unnormalized, i.e., it does not, by itself, allow judging whether a classifier is indeed useful (*sia davvero utile*) (e.g., better than decisions taken without the classifier)

Ideally we would like our system to provide optimal performance on the application data. However, we cannot measure a priori the performance of the system on the application data, since typically we do not have neither of the data nor the corresponding labels.

We can, on the other hand, employ an evaluation set to compute the performance of different models on the evaluation set itself.

The evaluation metrics thus:

- measure exactly our performance on the given dataset.
- can be used to obtain an estimate of our performance on application data that behaves similarly to our evaluation set.

Note that it is important that our metrics do not depend on the evaluation set empirical prior, when this may differ from the application prior.

The confusion matrix provides a complete summary of a classifier's predictions on a given dataset. For each combination of true class label C_j and predicted label C_i, it reports the number of samples from class C_j that were predicted as C_i. This includes both correct classifications (on the diagonal) and misclassifications (off the diagonal).

	Class C_1	Class C_2	$\cdots$	Class C_K
Prediction C_1	# samples of C_1 pred. as C_1	# samples of C_2 pred. as C_1	$\cdots$	# samples of C_K pred. as C_1
Prediction C_2	# samples of C_1 pred. as C_2	# samples of C_2 pred. as C_2	$\cdots$	# samples of C_K pred. as C_2
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
Prediction C_K	# samples of C_1 pred. as C_K	# samples of C_2 pred. as C_K	$\cdots$	# samples of C_K pred. as C_K

For a binary problem, we define the confusion matrix, per-class error rates and derived formulas:

	Class H_F	Class H_T
Prediction H_F	True Negative (TN)	False Negative (FN)
Prediction H_T	False Positive (FP)	True Positive (TP)

$$\text{acc} = \frac{TN + TP}{TN + FP + TP + FN}, \quad \text{err} = \frac{FP + FN}{TN + FP + TP + FN}$$

$$P_{fn} = \frac{FN}{FN + TP} \quad \text{(False Negative Rate — error rate for the positive class)}$$

$$P_{fp} = \frac{FP}{FP + TN} \quad \text{(False Positive Rate — error rate for the negative class)}$$

$$TPR = \frac{TP}{FN + TP} = 1 - FNR \quad \text{(True Positive Rate / recall / sensitivity)}$$

$$TNR = \frac{TN}{FP + TN} = 1 - FPR \quad \text{(True Negative Rate / specificity)}$$

We notice that all these metrics are calculated using only the samples from a single class at a time. Because of this, their values do not depend on how many samples there are in each class (the class proportions), but only on the performance within that specific class.

The proportion of samples for each class is the fraction (or percentage) of samples in your dataset that belong to each class.

We define the empirical class prior (computed on the evaluation set):

$$\pi_E^{emp} = \frac{TP + FN}{TN + FP + TP + FN}$$

The error rate can be represented as a weighted sum of the error rates of each class, where the weights depend on the empirical class prior:

$$\text{err} = \frac{FP + FN}{TN + FP + TP + FN} = \frac{FP}{TN + FP} \cdot \frac{TN + FP}{TN + FP + TP + FN} + \frac{FN}{TP + FN} \cdot \frac{TP + FN}{TN + FP + TP + FN}$$

$$= P_{fp}(1 - \pi_E^{emp}) + P_{fn}\,\pi_E^{emp}$$

In real-world scenarios, the class proportions (application prior) can be very different from those in the evaluation dataset. Creating a dataset that matches the real-world class proportions is often impractical, especially when one class is very rare. However, we can estimate the total error rate we would expect in the real scenario, by combining the per-class error rates (measured on a balanced dataset) with the real class priors π, using the error rate formula:

$$\text{err}_{app} = P_{fp}(1 - \pi) + P_{fn}\,\pi$$

This allows us to predict performance under real conditions without collecting a large number of rare class samples.

The error rate does not account for the cost of different errors.

The concept of cost is not restricted to a monetary cost, but is a generalization that should quantify the effects of an incorrect classification, and in particular reflect the relative effects of different error types. Typically, costs will change from application to application.

We want a metric that overcomes all these issues, allowing us to summarize the performance of a classifier with a single number that:

- depends on the application, rather than on the empirical prior
- accounts for the different costs of classification errors
- allows comparing different classifiers
- is normalized

A **normalized metric** is a performance measure that has been scaled or transformed to take values within a standard range, typically between 0 and 1, and allows for fair comparison.

Given a task, the goal of a classifier is to allow us to choose a suitable action (or decision) a to perform among a set of possible actions $\mathcal{A}$.

We can associate to each action a cost $C(a|k)$ that we have to pay when we choose action a and the sample belongs to class k.

We assume that a classifier R is able to produce a decision, labeling in our case, for each sample according to some rule.

We denote with $\mathcal{A}$ the set of actions corresponding to labeling a sample.

We denote with $a(x, R)$ the decision made by R for sample x, whose correct class is c.

The cost of such decision is $C(a(x, R), |, c)$.

Let's remind the definition of the expectation of a function as:

$$E[f(X)] = \int f(x)\, p(x)\, dx$$

i.e., it is the cost of predicting label $a(x, R)$ when the correct value is c.

We can express the Bayes risk of decisions made by our classifier for the target application population, i.e., the cost we expect to pay for using our system on the application population:

$$B = E_{X,C|E}[C(a(x,R)|c)]$$

where E denotes the application population, that is the real population on which the system will be used, and can also be interpreted as an evaluator who has complete knowledge of the application data.

X and C are random variables and are distributed according the real application distribution:

$$X, C|E$$

Unfortunately we do not have knowledge of the distribution $f_{X,C|E}$.

If we assume we know the application prior probabilities $\pi_c = P(C = c \mid E)$, we can express the Bayes risk for the target application as:

$$B = \sum_{c=1}^{K} \pi_c \int f_{X|C,E}(x|c)\, C(a(x,R)|c)\, dx = \sum_{c=1}^{K} \pi_c\, E_{X|C,E}[C(a(x,R)|c) \mid c]$$

where the distribution $f_{X|C,E}(x|c)$ is the class likelihood of the application population.

Since we do not have access to $f_{X|C,E}(x|c)$, we cannot compute the Bayes risk. However, if we have a set of labeled evaluation samples $(x_1, c_1), \ldots, (x_N, c_N)$, then we can approximate the expectations by averaging the cost over the samples. Indeed, if samples x_i are generated by $X|C, E$, as the number of samples per class becomes large we have that:

$$E_{X|C,E}[C(a(x,R)|c) \mid c] = \int C(a(x,R)|c) f_{X|C,E}(x|c) dx \approx \frac{1}{N_c} \sum_{i|c_i=c} C(a(x_i,R)|c)$$

i.e., the integral can be approximated by the average cost computed over samples of each class of the evaluation set.

We can finally define the empirical Bayes risk as:

$$B_{emp} = \sum_{c=1}^{K} \pi_c \frac{1}{N_c} \sum_{i|c_i=c} C(a(x_i,R)|c)$$

The risk measures the costs of our decisions for a target application over the evaluation samples.

We can use B_{emp} to compare recognizers. Lower costs correspond to better performance.

If we set the class prior probabilities π_c equal to the empirical class priors π_c^{emp} from the evaluation set, then the empirical Bayes risk corresponds to the total misclassification cost on the evaluation samples. This is similar to the error rate, but it also takes into account the specific costs associated with different types of errors.

To compute the empirical Bayes risk, we need the confusion matrix, the cost matrix, and the class prior probabilities.

Let's consider a binary problem.

	Class H_F	Class H_T
Prediction H_F	$C(H_F \mid H_F) = 0$	$C(H_F \mid H_T) = C_{fn}$
Prediction H_T	$C(H_T \mid H_F) = C_{fp}$	$C(H_T \mid H_T) = 0$

C_{fn} is the cost of false negative errors, C_{fp} is the cost of false positive errors.

Let c_i^* be the predicted label for sample x_i, whose label is c_i. The empirical Bayes risk is:

$$B_{emp} = \pi_T \frac{1}{N_T} \sum_{i|c_i=H_T} C(c_i^* \mid H_T) + (1-\pi_T) \frac{1}{N_F} \sum_{i|c_i=H_F} C(c_i^* \mid H_F)$$

$$= \pi_T C_{fn} P_{fn} + (1-\pi_T) C_{fp} P_{fp}$$

B_{emp} is also called unnormalized Detection Cost Function:

$$DCF_u(\pi_T, C_{fn}, C_{fp}) = \pi_T C_{fn} P_{fn} + (1-\pi_T) C_{fp} P_{fp}$$

The normalized DCF is used to evaluate how much better our classifier is compared to a trivial solution.

Dummy systems are classifiers that always predict the same class, either always accepting:

$$P_{fp} = 1, \quad P_{fn} = 0 \implies DCF_u = (1 - \pi_T)C_{fp}$$

or always rejecting:

$$P_{fp} = 0, \quad P_{fn} = 1 \implies DCF_u = \pi_T C_{fn}$$

Dummy systems are based only on the prior probabilities and costs, without looking at the data. The best dummy system is the one with the lowest cost among these options.

We compute the DCF for our real system and compare it to the cost of the best dummy system:

$$DCF(\pi_T, C_{fn}, C_{fp}) = \frac{DCF_u(\pi_T, C_{fn}, C_{fp})}{\min(\pi_T C_{fn}, (1 - \pi_T)C_{fp})}$$

By normalizing in this way, we can see if our classifier is truly extracting useful information from the data: a normalized DCF close to 1 means our system is no better than a dummy, while a value close to 0 means our system is much better.

The normalized DCF is invariant to scaling of the costs. This means that if we multiply all the costs by a constant factor, the value of the normalized DCF does not change. As a result, we can always rescale the problem so that the costs are uniform (for example, both set to 1), and the effect of the original costs and class priors is absorbed into a new, effective prior probability. This effective prior summarizes the combined influence of the original priors and costs.

Similarly, we can also transform the problem so that the prior is uniform (for example, 0.5 for each class), and the effect of the original prior is absorbed into new, effective classification costs.

In both cases, the normalized DCF remains the same, and the different formulations are equivalent in terms of evaluating classifier performance.

This property allows us to fairly compare classifiers and applications with different priors and costs, since the normalized DCF reflects only the true ability of the classifier to extract useful information from the data, independent of the original cost and prior settings.

For multiclass tasks, the evaluation is more complex, and we cannot represent any application with a single parameter (the effective prior).

However, we can compute the empirical Bayes risk, and a normalized DCF. The latter is obtained by scaling the empirical Bayes risk by the cost of the best dummy system.

Let's consider a binary classification problem.

In many cases, binary classifiers output a single score that measures the strength of the hypothesis for one class (e.g., the log-likelihood ratios in generative models, the posterior log-probability ratio in discriminative models, or the score from non-probabilistic models like SVM).

A higher score indicates a stronger preference for a particular class.

Decisions can be cast as comparing the score with a threshold. By varying the threshold, we obtain different error rates for the two classes.

We can visualize the performance of the classifier for different thresholds by plotting the error rates as a function of the threshold. The threshold at which the error rates for the two classes are equal, $P_{fn} = P_{fp}$, is called the Equal Error Rate.

To better understand the trade-off between different types of errors as we change the threshold, we use graphical tools such as the ROC curve (Receiver Operating Characteristic), which plots the TPR against the FPR, and the DET curve (Detection Operating Characteristic), which plots the FNR against the FPR.

These curves help us evaluate and compare the performance of classifiers across all possible thresholds.

Since the error rates depend on the selected threshold, we would like to optimize the threshold selection for a given application. More generally, we seek a principled way to transform the classifier scores into effective decisions.

For now, let's assume our classifier is probabilistic, meaning it can provide class posterior probabilities for a given sample x in a specific application $P(C = k \mid x, R)$. This allows us to compute the expected cost of action a according to the posterior probabilities $P(C = k \mid x, R)$:

$$C_{x,R}(a) = E_{C|X,R}[C(a|k) \mid x, R] = \sum_{k=1}^{K} C(a|k)P(C = k \mid x, R)$$

This expected cost measures the cost that we expect to pay given the recognizer knowledge, encoded through the class distribution $P(C = k \mid x, R)$.

The optimal Bayes decision consists in choosing the action that minimizes the expected cost:

$$a^*(x, R) = \arg\min_a C_{x,R}(a)$$

This represents the action that will result in the lower expected cost, according to the recognizer beliefs.

Note that different recognizers may have different posterior beliefs, and thus may provide different decisions. Optimal Bayes decisions are optimal from the point of view of the recognizer.

If the evaluator and the recognizer agree, meaning they report the same class posterior probabilities for any sample, the Bayes decisions minimize the Bayes risk.

Indeed, we can express the Bayes risk as:

$$B = E_{X,C|E}[C(a(x, R)|c)] = \int f_{X|E}(x) \sum_{c=1}^{K} C(a(x, R)|c) P(C|X = x, E) dx$$

$$= \int f_{X|E}(x) E_{C|X,E}[C(a(x, R)|c) \mid x] dx$$

If $C|X, R \sim C|X, E$, then the risk becomes:

$$B = \int f_{X|E}(x) E_{C|X,R}[C(a(x, R)|c) \mid x] dx \geq \int f_{X|E}(x) E_{C|X,R}[C(a^*(x, R)|c) \mid x] dx$$

since, for any x, $a^*(x, R)$ is the action that minimizes the expected cost:

$$C_{x,R}(a(x, R)) \geq C_{x,R}(a^*(x, R)), \ \forall x$$

Optimally can be interpreted in two ways. From the point of view of the recognizer R, optimal Bayes decisions are those that minimize the Bayes risk

according to the recognizer's own knowledge, that is, based on its estimated posterior probabilities.

However, if we imagine a system that has complete knowledge of the true data distributions, an evaluator (E), then the optimal Bayes decisions would minimize the true Bayes risk as evaluated by this evaluator. In this ideal case, the Bayes risk represents the lowest possible cost we could achieve for classifying the test data. Of course, in practice, we do not have full knowledge of the data, so we cannot compute this true risk.

Note that the evaluator is an abstract entity that knows the real distributions and serves as a theoretical reference for the best possible performance.

optimal Bayes decisions of a system that has complete data knowledge $R = E$ minimize the Bayes risk of the evaluator E

Note that, even with complete knowledge, we may have $0 < P(C = k \mid X, E) < 1$ for different classes, e.g., when samples with the same feature values may have different labels, thus even in this case the Bayes risk may be non-zero.

Let's consider a binary problem with cost matrix:

	Class H_F	Class H_T
Prediction H_F	0	C_{fn}
Prediction H_T	C_{fp}	0

The optimal decision can be expressed as:

$$a^*(x, R) = \begin{cases} H_T & \text{if } C_{fp}P(H_F|x, R) < C_{fn}P(H_T|x, R) \\ H_F & \text{if } C_{fp}P(H_F|x, R) > C_{fn}P(H_T|x, R) \end{cases}$$

Equivalently, defining $r(x) = \log \dfrac{C_{fn}P(H_T|x, R)}{C_{fp}P(H_F|x, R)}$:

$$a^*(x, R) = \begin{cases} H_T & \text{if } r(x) > 0 \\ H_F & \text{if } r(x) < 0 \end{cases}$$

If R is a generative model for x, then we can express r in terms of costs, prior probabilities and likelihoods as:

$$r(x) = \log \frac{\pi_T C_{fn}}{(1-\pi_T)C_{fp}} \cdot \frac{f_{X|H,R}(x|H_T)}{f_{X|H,R}(x|H_F)}$$

The decision rule thus becomes:

$$r(x) \gtrless 0 \iff \log \frac{f_{X|H,R}(x|H_T)}{f_{X|H,R}(x|H_F)} \gtrless -\log \frac{\pi_T C_{fn}}{(1-\pi_T)C_{fp}}$$

The triplet (π_T, C_{fn}, C_{fp}) represents the working point of an application for a binary classification task. Note that, as for the Bayes empirical risk, the triplet is actually redundant, in the sense we can build equivalent applications $(\pi'_T, C'_{fn}, C'_{fp})$ which have the same decision rule as the original application, but different costs and priors.

For system producing well-calibrated log-likelihood ratios:

$$s = \log \frac{f_{X|C}(x|H_T)}{f_{X|C}(x|H_F)}$$

the optimal threshold (optimal Bayes decision) becomes, in terms of effective prior:

$$t = -\log \frac{\tilde{\pi}}{1-\tilde{\pi}}, \quad \text{where } \tilde{\pi} = \frac{\pi_T C_{fn}}{\pi_T C_{fn} + (1-\pi_T)C_{fp}}$$

A quantity (such as a log-likelihood ratio) is well-calibrated if its predicted values correspond closely to the true probabilities or frequencies observed in reality. For example, a well-calibrated probability estimate of 70% should be correct about 70% of the time.

Note that LLRs allow disentangling the classifier from the application.

In general, systems often do not produce well-calibrated LLRs. If so, we say that scores are mis-calibrated.

For a given application, we can measure the additional cost due to the use of mis-calibrated scores. We can define the minimum cost DCF_{min} corresponding to the use of the optimal threshold for a given evaluation set. We consider varying the threshold t to obtain all possible combinations of P_{fn} and P_{fp} for the evaluation set. We select the threshold corresponding to the lowest DCF. The corresponding value DCF_{min} is the cost we would pay if

we knew before-hand the optimal threshold for the evaluation set. We can think of this value as a measure of the quality of the classifier. We can also compute the actual DCF obtained using the threshold corresponding to the effective prior $\tilde{\pi}$. The difference between the actual and minimum DCF represents the loss due to score mis-calibration.

To reduce mis-calibration, various calibration strategies can be adopted.

A simple approach is to use a validation set to find an optimal threshold for a specific application. However, more general methods look for functions that transform classifiers scores into approximately well-calibrated LLRs, in a way that is independent of the target application.

We want to compute a transformation function f that maps the classifiers scores s to well-calibrated scores $s_{cal} = f(s)$.

Two common approaches for score calibration are **isotonic regression** and **score models**.

Isotonic regression is a non-linear, monotonic transformation that provides optimal calibration for the data it is trained on. This method produces a piecewise non-linear function, which may require interpolation for unseen scores and does not allow extrapolation outside the range of training scores. It can also be computationally expensive when the calibration set is large.

Score models, such as prior-weighted logistic regression, approximate the transformation achieved by isotonic regression. These models require assumptions about the calibration transformation or the distribution of class scores. They are estimated on a training set and allow for extrapolation beyond the range of observed scores. Typically, they are fast to evaluate, but may provide good calibration only for a limited range of operating points.

7 Logistic Regression

Logistic regression: Logistic regression is a widely used statistical and machine learning method for binary classification tasks. It models the probability that a given input belongs to a particular class by applying the logistic (sigmoid) function to a linear combination of the input features. Logistic regression is particularly valued for its simplicity, interpretability, and ability to provide well-calibrated probability estimates, making it a fundamental tool in both statistics and machine learning.

Logistic regression is a discriminative approach for classification. We directly model the class posterior distribution $P(C|X)$.

Being discriminative means it focuses on finding the boundary between classes, not on modeling the distribution of the features themselves.

Let's consider a binary problem. Our approach is to assume that our decision rule should be a linear rule represented by w, b. Given w and b, we can compute the expression for the posterior class probability as:

$$P(C = h_1|x, w, b) = e^{w^T x+b} P(C = h_0|x, w, b)$$

$$e^{w^T x+b} + 1 = P(C = h_1|x, w, b)$$

Solving for $P(C = h_1|x, w, b)$ we obtain:

$$P(C = h_1|x, w, b) = \frac{e^{w^T x+b}}{1 + e^{w^T x+b}} = \frac{1}{1 + e^{-(w^T x+b)}} = \sigma(w^T x + b)$$

where

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

is called sigmoid function or logistic function.

Some useful properties of σ are:

$$\sigma(1 - x) = \sigma(-x)$$

$$\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$$

The expression $P(C = h_1|x, w, b) = \sigma(w^T x + b)$ provides a model that allows computing the posterior probabilities for h_1 and h_0.

The model assumes that decision rules are linear surfaces (hyperplanes) orthogonal to w.

[INTEGRAZIONE TEORICA]
Gli iperpiani sono generalizzazioni dei piani nello spazio a più dimensioni. In uno spazio tridimensionale, un piano è una superficie piatta a due dimensioni. In uno spazio a (n) dimensioni, un iperpiano è una "superficie" piatta di dimensione (n-1) che divide lo spazio in due parti.

The model parameters are $\theta = (w, b)$.

Obviously, if we knew w, b then we could compute the predictive distribution for the class labels $P(C = h_1|x, w, b)$. But we do not know them.

We are interested in finding an alternative way to estimate an effective classification rule that does not require an explicit model of the feature vectors distribution. So, we concentrate directly on the form of the class posterior probabilities.

We follow a frequentist approach, so we compute an estimate for w and b from a set of n training samples.

We assume we have a labeled dataset $D = \{(x_1, c_1), \ldots, (x_n, c_n)\}$.

We assume that feature vectors and corresponding class labels are i.i.d. given the model parameters.

The model parameters are $\theta = (w, b)$.

The complete-data likelihood for θ consists of the joint density of the observed training set variables, given parameter vector θ:

$$L(\theta) = f_{X_1 \ldots X_n, C_1 \ldots C_n}(x_1, ..., x_n, c_1, ..., c_n|\theta)$$

Since we assume i.i.d. observations, we can factorize the likelihood as:

$$L(\theta) = f_{X_1 \ldots X_n, C_1 \ldots C_n}(x_1, ..., x_n, c_1, ..., c_n|\theta) = \prod_{i=1}^{n} f_{X,C}(x_i, c_i|\theta)$$

The difference with respect to generative models is in the way we represent the joint density:

$$f_{X,C}(x_i, c_i|\theta) = P(C = c_i|X = x_i, \theta) \cdot f_X(x_i)$$

Note that, since classification requires computing posterior class probabilities, and we are defining an explicit model for such terms, we do not need to explicitly provide an expression for the marginal feature density $f_X(x)$.

So, the complete-data log-likelihood becomes:

$$\log L(\theta) = \log \prod_{i=1}^{n} f_{X,C}(x_i, c_i|\theta) = \sum_{i=1}^{n} \log P(C_i = c_i|X_i = x_i, \theta) + \sum_{i=1}^{n} \log f_X(x_i)$$

We can estimate the model parameters by following a ML approach:

$$\hat{\theta}_{ML} = \arg\max_{\theta} \log L(\theta)$$

Since the model parameters θ influence only the first sum, optimization of the log-likelihood corresponds to the maximization of the conditional probability of the observed dataset labels, given the observed feature vectors and the model parameters:

$$\ell(\theta) = \sum_{i=1}^{n} \log P(C_i = c_i|X_i = x_i, \theta)$$

We now need to express our model in the expression for $\ell(\theta)$. We assume that the label for class h_1 is 1 and the label for class h_0 is 0.

Our model specifies the probabilities for observed class labels in terms of w and b:

$$y_i = P(C_i = 1|x_i, w, b) = \sigma(w^T x_i + b)$$

It follows that:

$$P(C_i = 0|x_i, w, b) = 1 - y_i = 1 - \sigma(w^T x_i + b) = \sigma(-(w^T x_i + b))$$

We note that $C_i|x_i, w, b$ follows a Bernoulli distribution:

$$C_i|x_i, w, b \sim Ber(\sigma(w^T x_i + b)) = Ber(y_i)$$

The conditional probability of the class labels, ie our objective function, is thus given by:

$$\ell(w, b) = \log \prod_{i=1}^{n} P(C_i = c_i|x_i, w, b) = \log \prod_i y_i^{c_i}(1-y_i)^{1-c_i} = \sum_{i=1}^{n} [c_i \log y_i + (1-c_i) \log(1-y_i)]$$

Our goal is the maximization of ℓ, which corresponds to the maximization of the likelihood function, ie the ML solution. We thus seek w, b that maximize $\ell(w, b)$:

$$\hat{w}, \hat{b} = \arg\max_{w,b} \ell(w, b) = \arg\max_{w,b} \sum_{i=1}^{n} [c_i \log y_i + (1 - c_i) \log(1 - y_i)]$$

The ML solution is also the solution that minimizes the average cross-entropy between the distribution of observed and predicted labels.

Cross-entropy is a measure of the difference between two probability distributions: the true distribution (the observed labels) and the predicted distribution (the model's output probabilities). In classification, the average cross-entropy is the mean value of the cross-entropy computed over all samples in your dataset. Minimizing the average cross-entropy means making the predicted probabilities as close as possible to the true labels.

Rather than maximizing $\ell(w, b)$, we can minimize:

$$J(w, b) = -\ell(w, b) = -\sum_{i=1}^{n} [c_i \log y_i + (1 - c_i) \log(1 - y_i)]$$

The expression:

$$H(c_i, y_i) = -[c_i \log y_i + (1 - c_i) \log(1 - y_i)]$$

represents the binary cross-entropy between the distribution of observed and predicted labels for the i-th sample.

More in general, let P and Q be two distributions over the same domain, the cross-entropy between the two distributions is defined as:

$$H(P, Q) = E_P[-\log Q(x)]$$

For discrete distributions, this can be expressed as:

$$H(P, Q) = \sum_{x \in S} P(x)(-\log Q(x))$$

In our case, P is the empirical distribution of class labels, from the point of view of an observer who knows the actual label:

$$P(C_i = 1 | X_i = x_i, E) = \begin{cases} 1 & \text{if } c_i = 1 \\ 0 & \text{if } c_i = 0 \end{cases}$$

$$P(C_i = 0 | X_i = x_i, E) = \begin{cases} 0 & \text{if } c_i = 1 \\ 1 & \text{if } c_i = 0 \end{cases}$$

or, equivalently:

$$P(C_i = 1 | X_i = x_i, E) = c_i, \quad P(C_i = 0 | X_i = x_i, E) = 1 - c_i$$

ie a Bernoulli distribution with parameter c_i.

Distribution Q is the distribution for the predicted labels according to our recognizer R:

$$Q(c) = P(C_i = c | X_i = x_i, R(w, b))$$

i.e.:

$$Q(1) = P(C_i = 1|X_i = x_i, R(w, b)) = y_i = \sigma(w^T x_i + b)$$

$$Q(0) = P(C_i = 0|X_i = x_i, R(w, b)) = 1 - y_i = 1 - \sigma(w^T x_i + b)$$

Logistic regression looks for the minimizer of the average cross-entropy between the distribution for the training set labels of an evaluator E who knows the real label and the distribution for the training set labels as predicted by the model $R(w, b)$ itself.

The cross-entropy, as a function of Q, is minimized when $Q = P$.

The cross-entropy can also be interpreted as a measure of the difference between P and Q. In our case, it measures how different the predicted distribution $\text{Ber}(y_i)$ is from the empirical label distribution $\text{Ber}(c_i)$ (the distribution of the evaluator E).

Minimizing the average cross-entropy means we are looking for conditional label distributions as similar, on average, as possible to the empirical one, given the model constraints (ie a linear classification rule).

Alternatively, we can regard the process as maximization of the likelihood for the observed labels.

Another interesting interpretation of the average cross-entropy can be obtained by rewriting the cross-entropy in terms of $z_i = 2c_i - 1$. This terms z_i still represent class labels:

$$z_i = \begin{cases} 1 & \text{if } c_i = 1 \\ -1 & \text{if } c_i = 0 \end{cases}$$

The objective function that we want to minimize corresponds to:

$$J(w, b) = \sum_i H(c_i, y_i)$$

where:

$$H(c_i, y_i) = -[c_i \log y_i + (1 - c_i) \log(1 - y_i)]$$

Note that $H(c_i, y_i)$ is a function of c_i, but also of w, b and x_i, since $y_i = \sigma(w^T x_i + b)$.

Let $s_i = w^T x_i + b$. In terms of z_i we can rewrite H as:

$$H(c_i, y_i) = -[c_i \log y_i + (1 - c_i) \log(1 - y_i)]$$

$$\begin{cases} -\log \sigma(s_i) & \text{if } c_i = 1 \\ -\log(1 - \sigma(s_i)) = -\log \sigma(-s_i) & \text{if } c_i = 0 \end{cases}$$

$$H(c_i, y_i) = -\log \sigma(z_i s_i) = -\log \sigma(z_i(w^T x_i + b))$$

$$= \log(1 + e^{-z_i(w^T x_i + b)})$$

The objective function can thus be rewritten as:

$$J(w, b) = \sum_{i=1}^{n} H(c_i, y_i) = \sum_{i=1}^{n} \log(1 + e^{-z_i(w^T x_i + b)}) = \sum_{i=1}^{n} \ell(-z_i(w^T x_i + b))$$

where:

$$\ell(x) = \log(1 + e^{-x})$$

is the logistic function.

Our goal is to find the minimizer of $J(w, b)$.

We can interpret the function ℓ as the cost of the prediction made with model w, b for each sample.

Remember that the log-posterior class probability ratio for sample x_i is:

$$\log \frac{P(C_i = 1 | X_i = x_i)}{P(C_i = 0 | X_i = x_i)} = w^T x_i + b = s_i$$

The decision rule takes the form $s_i \gtrless 0$. Since $s_i = w^T x_i + b$, decision rules are linear hyperplane orthogonal to the vector w.

s_i is related to the distance of the sample x_i from the separating surface. When s_i is positive, our classifier is favoring class h_1, whereas negative s_i means we are classifying the sample as belonging to class h_0.

The cost we pay for each sample is $\ell(-z_i s_i)$.

If the prediction and the actual class agree, ie $(z_i = 1, s_i > 0)$ or $(z_i = -1, s_i < 0)$, then $-z_i s_i < 0$ and we pay a low cost. The cost becomes exponentially smaller (asymptotically) as the absolute value of s_i increases (we move away from the separation surface).

If the prediction and the actual class disagree, ie $(z_i = 1, s_i < 0)$ or $(z_i = -1, s_i > 0)$, then $-z_i s_i > 0$ and we pay a cost that increases (asymptotically) linearly with $|s_i|$.

We can thus interpret the logistic regression objective as a measure of an empirical (because it's computed on the observed samples) risk. Our goal is to minimize the empirical risk.

More in general, empirical risk minimization is a framework for the estimation of classification models which aims at minimizing an empirical risk function over our training data. So, the generalized risk minimization problem is to minimize the risk:

$$R(\theta) = \sum_i \ell(\theta, x_i, z_i)$$

where ℓ is called loss or cost function, and θ are the parameters of the classification model, e.g. (w, b) in our case.

Logistic regression solutions cannot be computed in closed form, so we will resort to numerical solvers through an algorithm that requires a function that computes the loss and its gradient with respect to w and b.

If classes are linearly separable, the logistic regression solution is not defined because the loss can always be made smaller by increasing the weights, and the parameters do not converge to a finite value. Basically, we can make the values of s_i arbitrarily high by simply increasing the norm of w and changing accordingly the value of b. As we increase the norm of w, the loss becomes lower, thus we are decreasing the objective function. The function does not have a minimum, but has an infimum $\inf J(w, b) = 0$, corresponding to $|w| \to \infty$.

To make the problem solvable again, we can look for solutions with small

norms by introducing a norm penalty to the objective function. The penalty is called the regularization term. The objective function that we minimize is:

$$R(w,b) = \frac{\lambda}{2}\|w\|^2 + \sum_{i=1}^{n} \log(1 + e^{-z_i(w^T x_i + b)})$$

where λ is a hyper-parameter that allows specifying the relative weight of the regularization term.

Alternatively, we look for the minimizer of:

$$R(w,b) = \frac{\lambda}{2}\|w\|^2 + \frac{1}{n}\sum_{i=1}^{n} \log(1 + e^{-z_i(w^T x_i + b)})$$

where the risk is averaged over all samples, and λ is the regularization coefficient.

λ is a hyper-parameter, and should be selected to optimize the performance of the classifier. Note that λ cannot be computed by minimizing R with respect to λ, as we would obtain the trivial solution $\lambda = 0$.

The selection of good values for λ should thus be based on other approaches, such as cross-validation.

The model is called regularized Logistic Regression, and is an example of a regularized risk minimization problem:

$$R(w,b) = \Phi(w,b) + \frac{1}{n}\sum_{i=1}^{n} \ell(x_i, z_i, w, b)$$

The regularization term Φ, in our case $\frac{\lambda}{2}|w|^2$, can be interpreted as a term that favors simpler solutions.

Regularization allows reducing the risk of over-fitting the training data. If λ is too large, we will obtain a solution that has a small norm, but is not able to well separate the classes. If λ is too small, we will get a solution that has good separation on the training set, but may have poor generalization.

Note that the non-regularized model is invariant to linear transformations of the feature vectors. On the other hand, the regularized version of the model

is not invariant. It is therefore useful, in some cases, to pre-process data so that dynamic ranges of different features are similar. Cross-validation can help identify good pre-processing strategies.

Common preprocessing strategies that may be worth trying are: center the data $x_i' = x_i - \mu$, standardize variances, whitening the covariance matrix (ie normalize variances while making features uncorrelated), length normalization $x_i' = \frac{x_i}{|x_i|}$.

The logistic regression score can be interpreted as the logarithm of the ratio between class posterior probabilities. The model parameters have been trained to optimize the probability of the training set labels. Therefore, the model will implicitly reflect the empirical class prior of the training set. The model posterior probabilities are thus suited for applications whose effective prior is close to the empirical training set prior, but may provide poor performance for different applications.

To soften this issue, we can recover a score that behaves like a log-likelihood ratio by subtracting from the score s the empirical prior log-odds of the training set $\log \frac{n_T}{n_F}$:

$$s_{llr} = w^T x + b - \log \frac{n_T}{n_F} = w^T x + b - \log \frac{\pi_{Temp}}{1 - \pi_{Temp}}$$

We can then use s_{llr} as a log-likelihood ratio. For a given application $(\pi_T, 1 - \pi_T, C)$ we can compute decisions by comparing $s_{llr} \gtrless \log \frac{\pi_T}{1-\pi_T}$.

While this allows for effective decisions for the model w, b, the orientation of w is still optimized for the training set empirical prior. Changing the threshold allows us to select a better decision rule, but only among hyperplanes that are orthogonal to w.

If we knew the application prior π_T before training the model, then it may be preferable to directly optimize the separation rule for the target application prior. The model is sometimes called prior-weighted logistic regression:

$$R(w) = \frac{\lambda}{2}\|w\|^2 + \frac{\pi_T}{n_T} \sum_{i:z_i=1} \ell(-z_i s_i) + \frac{1 - \pi_T}{n_F} \sum_{i:z_i=-1} \ell(-z_i s_i)$$

The standard logistic regression model corresponds to the prior-weighted logistic regression model trained with the training set empirical prior π_T^{emp}.

Also in this case, we can compute:

$$s_{llr} = w^T x + b - \log \frac{\pi_T}{1 - \pi_T}$$

Regularization is important, especially when we do not reduce the dimensionality, since we have more parameters to estimate, so over-fitting is more severe. If we regularize too much, the model performs poorly again.

We now consider a problem with K classes, labeled from 1 to K.

To extend the Logistic Regression model to multiclass tasks we start again from the form of the posterior probabilities of the linear Gaussian classifier with uniform priors:

$$\log P(C = j|x) = w_j^T x + b_j - k(x)$$

where $k(x)$ collects terms that depend on the sample x, but not on the class j. It follows that, as a function of the class:

$$P(C = j|x) \propto e^{w_j^T x + b_j}$$

Since we have a closed-set classification problem, then:

$$\sum_{j=1}^{K} P(C = j|x) = 1$$

The normalization factor for $P(C = j|x)$ is thus $\sum_{j=1}^{K} e^{w_j^T x + b_j}$ and the posterior probability corresponds to:

$$P(C = k|x) = \frac{e^{w_k^T x + b_k}}{\sum_j e^{w_j^T x + b_j}}$$

The function $f(s) = \left(\frac{e^{s_1}}{\sum_j e^{s_j}}, \ldots, \frac{e^{s_K}}{\sum_j e^{s_j}} \right)$ is called softmax.

Given the model parameters:

$$W = [w_1, ..., w_K], \quad b = [b_1, ..., b_K]^T$$

the logistic regression model allows computing the probability of each class:

$$P(C = k|W, b, x) = \frac{e^{w_k^T x + b_k}}{\sum_j e^{w_j^T x + b_j}}$$

We can express the log-probability of the training class labels as:

$$\ell(W, b) = \sum_{i=1}^{n} \log P(C_i = c_i|X_i = x_i, W, b)$$

If we consider sample x_i, its class posterior distribution is thus a categorical distribution:

$$C_i|W, b, X_i = x_i \sim Cat(y_i)$$

where:

$$y_{ik} = \frac{e^{w_k^T x_i + b_k}}{\sum_j e^{w_j^T x_i + b_j}}$$

Remember that the categorical density $P(C_i = c_i|X_i = x_i, W, b)$ can be expressed using a 1-of-K encoding, as:

$$\log P(C_i = c_i|X_i = x_i, W, b) = \log P(C_i = c_i|y_i) = \sum_{k=1}^{K} z_{ik} \log y_{ik}$$

where z_i is a vector that has all components equal to 0, except for the index c_i which is equal to 1.

The labels log-probability can thus be expressed as:

$$\ell(W, b) = \sum_{i=1}^{n} \sum_{k=1}^{K} z_{ik} \log y_{ik}$$

The terms $y_{ik} = \frac{e^{w_k^T x_i + b_k}}{\sum_j e^{w_j^T x_i + b_j}}$ represent the distribution for the class labels according to the Logistic Regression model.

The expression:

$$H(z_i, y_i) = -\sum_{k=1}^{K} z_{ik} \log y_{ik}$$

represents the multiclass cross-entropy between the observed and predicted label distributions for sample x_i.

We estimate W and b as to maximize the likelihood for the training labels. The ML solutions is again the solution that minimizes the average cross-entropy:

$$\arg\max_{W,b} \ell(W, b) = \arg\max_{W,b} \sum_{i=1}^{n} H(z_i, y_i) = \arg\min_{W,b} \sum_{i=1}^{n} H(z_i, y_i)$$

Compared to the binary case, the model is over-parametrized, ie we can add a constant vector to all terms w_i without changing the model.

Finally, we can cast the problem as a minimization of a loss function. We rewrite the objective in terms of class labels c as:

$$J(W, b) = -\sum_{i=1}^{n} \sum_{k=1}^{K} z_{ik} \log y_{ik}$$

$$= -\sum_{i=1}^{n} \log \frac{e^{w_{c_i}^T x_i + b_{c_i}}}{\sum_{c=1}^{K} e^{w_c^T x + b_c}}$$

$$= \sum_{i=1}^{n} [\log \sum_{c=1}^{K} e^{w_c^T x + b_c} - w_{c_i}^T x_i - b_{c_i}]$$

$$= \sum_{i=1}^{n} \ell(x_i, c_i, W, b)$$

where ℓ is also called softmax loss.

We can add a regularization term to reduce over-fitting. We thus look for the minimizer of:

$$R(W, b) = \Phi(W) + \frac{1}{n} J(W, b)$$

We can replace the average cross-entropy with prior-weighted average cross entropy to account for priors that are different from the empirical training set prior.

Logistic regression can be applied not only to the original feature space, but also to an expanded feature space obtained by transforming the input features through a mapping $\phi(x)$. By choosing an appropriate transformation, it is possible to make the classes approximately linearly separable in the new space, even if they are not separable in the original space. In this expanded space, logistic regression computes linear separation rules for the transformed features, which correspond to non-linear separation surfaces (such as quadratic boundaries) in the original feature space. However, increasing the complexity of the transformation can rapidly increase the dimensionality of the feature space, which may lead to higher computational costs and a greater risk of overfitting.

We can thus train a Logistic Regression model using feature vectors $\phi(x)$ rather than x. This space is also called expanded feature space. The Logistic Regression model (both for binary and multiclass) allows computing linear separation rules for the transformed features $\phi(x)$. Since expressions $w^T\phi(x) + c$ correspond to quadratic forms in the original feature space, we are actually estimating quadratic separation surfaces in the original space.

In general, we can consider a transformation $\phi(x)$ of our feature space such that our classes are approximately linearly separable in the expanded feature space. We have to pay attention that the dimensionality of the expanded feature space can grow very quickly.

8 Gaussian Mixture Models

Gaussian Mixture Models

Gaussian Mixture Models (GMMs) are probabilistic models used to represent data that is generated from a combination of several Gaussian distributions, each with its own mean and covariance. GMMs are widely used for clustering, density estimation, and as generative models in machine learning. By modeling complex data distributions as a mixture of simpler Gaussian components, GMMs can capture a wide variety of data patterns and are a fundamental tool in unsupervised learning and pattern recognition. GMMs are an alternative to model a generic distribution. They allow approximating any sufficiently regular distribution to a desired degree. Of course, since we are estimating the density from data, we require a sufficient amount of data to obtain good estimates. The use of GMMs is not restricted to classification. GMMs can be employed also in other tasks that require estimating a population density. For example, GMMs provide an alternative to K-means for clustering.

Let's consider the Gaussian classifier. The samples of each class are modeled by a Gaussian density:

$$f_{X|C}(x|c) = \mathcal{N}(x;\, \mu_c,\, \Sigma_c)$$

To compute class posterior probabilities we had to compute the marginal density $f_X(x)$. If the class prior probabilities are $P(C = c) = \pi_c,\ c = 1, \ldots, K$, then $f_X(x)$ is given by:

$$f_X(x) = \sum_{c=1}^{K} f_{X|C}(x|c)\, P(C = c) = \sum_{c=1}^{K} \pi_c \mathcal{N}(x;\, \mu_c,\, \Sigma_c) \tag{1}$$

Expression (1) is an example of K-components Gaussian Mixture Model:

$$X \sim GMM(\mathcal{M}, \mathcal{S}, \pi), \quad f_X(x) = \sum_{c=1}^{K} \pi_c \mathcal{N}(x;\, \mu_c,\, \Sigma_c)$$

More in general, a Gaussian Mixture Model is a density model obtained as a weighted combination of Gaussians:

$$f_X(x) = \sum_{c=1}^{K} w_c \mathcal{N}(x;\, \mu_c,\, \Sigma_c)$$

The distribution parameters are the components means:

$$\mathcal{M} = \{\mu_1, \ldots, \mu_K\}$$

the component covariances:

$$\mathcal{S} = \{\Sigma_1, \ldots, \Sigma_K\}$$

and the weights:

$$\mathbf{w} = \{w_1, \ldots, w_K\}$$

Remember that, for f_X to be a density, we need that its integral is equal to 1. Integrating we have:

$$\int f_X(x)\, dx = \int \sum_{c=1}^{K} w_c \mathcal{N}(x;\, \mu_c,\, \Sigma_c) dx = \sum_{c=1}^{K} w_c = 1$$

i.e., the weights must sum to 1:

$$\sum_{c=1}^{K} w_c = 1$$

Given a dataset $\mathcal{D} = \{x_1, \ldots, x_N\}$, we can thus assume that the samples have been independently generated by a GMM. We assume that random variables describing the samples X_i are i.i.d., with $X_i \sim \text{GMM}(\mathcal{M}, \mathcal{S}, \mathbf{w})$.

Note that we are considering a density estimation problem. If we are using GMMs for classification, $\mathcal{D}$ may correspond to the samples of a given class. However, in the following we do not assume any specific task, so that $\mathcal{D}$ is just a set of samples that we want to model by means of a GMM. In particular, we consider the dataset as unlabeled. $\mathcal{D}$

We can resort to ML to estimate the model parameters of the GMM that best describes the dataset $\mathcal{D}$. The ML estimation for GMMs is an ill-posed problem, ie it can lead to problematic or ill-defined solutions. Indeed, as long as we have more than 1 component, we can devise degenerate solutions for which the likelihood is not bounded above. But, the ML approach combined with heuristics to avoid degeneracy, provides good density estimates. We can write the likelihood for the model parameters as:

$$L = \prod_{i=1}^{N} f_{X_i}(x_i) = \prod_{i=1}^{N} GMM(x_i; \mathcal{M}, \mathcal{S}, \mathbf{w}) = \prod_{i=1}^{N} \sum_{c=1}^{K} w_c \mathcal{N}(x_i;\, \mu_c,\, \Sigma_c)$$

and the corresponding log-likelihood:

$$\log L = \sum_{i=1}^{N} \log \sum_{c=1}^{K} w_c \mathcal{N}(x_i;\, \mu_c,\, \Sigma_c)$$

A GMM can be interpreted as the marginal of a joint distribution of data points and corresponding clusters:

$$f_{X_i}(x_i) = \sum_{c=1}^{K} f_{X_i|C_i}(x_i|c) P(C_i = c) = \sum_{c=1}^{K} w_c \mathcal{N}(x_i;\, \mu_c,\, \Sigma_c)$$

Although our training set cannot be well modeled by a Gaussian distribution, we can imagine that the set can be partitioned into subsets (components or clusters), in such a way that the distribution of the points of each component can be modeled by a Gaussian probability density function. If we knew the component responsible for each sample, ie its cluster label, we could estimate the parameters of each Gaussian by ML from the points of each cluster. Unfortunately, in general the clusters are unknown. We treat cluster membership as an unobserved latent random variables. This means that the cluster membership of each sample is not observed in the data. Therefore, we treat cluster membership as a latent (hidden) random variable that must be estimated or inferred during model training. Intuitively, we want to estimate both cluster assignments and model parameters as to maximize the marginal distribution of the data.

Let's consider a set of GMM parameters $\theta = (\mathcal{M}, \mathcal{S}, \mathbf{w})$. The GMM defines a joint density of components (the clusters) and patterns. The density for sample x_i and component c is:

$$f_{X_i,C_i}(x_i, c) = w_c \mathcal{N}(x_i;\, \mu_c,\, \Sigma_c)$$

We can compute cluster (components) posterior probabilities:

$$\gamma_{c,i} = P(C_i = c | X_i = x_i) = \frac{f_{X_i,C_i}(x_i, c)}{f_{X_i}(x_i)} = \frac{w_c \mathcal{N}(x_i;\, \mu_c,\, \Sigma_c)}{\sum_{c'} w_{c'} \mathcal{N}(x_i;\, \mu_{c'},\, \Sigma_{c'})}$$

$\gamma_{c,i}$ are also called responsibilities.

As a first approximation, we might decide to assign a point to the cluster c with highest posterior probability $P(C_i = c \mid X_i = x_i)$. We thus associate a cluster label:

$$\hat{c}_i = \arg\max_c P(C_i = c | X_i = x_i)$$

to each sample. Given the cluster assignments, we can then estimate by the ML the new GMM parameters $\theta^{\text{new}} = (\mathcal{M}^{\text{new}}, \mathcal{S}^{\text{new}}, \mathbf{w}^{\text{new}})$.

We treat the cluster assignments as if they were known class labels. The log-likelihood is similar to that of a multivariate Gaussian classifier:

$$\sum_{i=1}^{N} \log f_{X_i|C_i}(x_i|\hat{c}_i) + \log P(C_i = \hat{c}_i) = \sum_{i=1}^{N} \log \mathcal{N}(x_i;\, \mu_{\hat{c}_i},\, \Sigma_{\hat{c}_i}) + \sum_{i=1}^{N} \log w_{\hat{c}_i}$$

and corresponds to a sum of two terms that depend on different subsets of the parameters:

$$\mathcal{L}_N(\mathcal{M}, \mathcal{S}) = \sum_{i=1}^{N} \log \mathcal{N}(x_i;\, \mu_{\hat{c}_i},\, \Sigma_{\hat{c}_i})$$

$$\mathcal{L}_C(\mathbf{w}) = \sum_{i=1}^{N} \log w_{\hat{c}_i} = \sum_{c=1}^{K} \sum_{i:\hat{c}_i=c} \log w_c$$

The ML solution is thus:

$$\hat{\mu}_c = \frac{1}{N_c} \sum_{i:\hat{c}_i=c} x_i, \quad \hat{\Sigma}_c = \frac{1}{N_c} \sum_{i:\hat{c}_i=c} (x_i - \hat{\mu}_c)(x_i - \hat{\mu}_c)^T$$

$$\hat{w}_c = \frac{N_c}{\sum_{c'=1}^{K} N_{c'}}$$

We could then obtain the updated set of model parameters $\theta^{\text{new}} = (\mathcal{M}^{\text{new}}, \mathcal{S}^{\text{new}}, \mathbf{w}^{\text{new}})$. We could iterate the process by computing new cluster assignments using θ^{new}, and using the updated assignments to update once again the model parameters, stopping when some criterion is met. There is a problem: we form hard clusters, ie a point is assigned to one and only one component of the GMM. This means that, in this approach, each point is assigned to exactly one component of the GMM—this is called "hard clustering." The problem with hard clustering is that it does not account for uncertainty: in reality, a point may belong to multiple components with different probabilities, especially when clusters overlap. Hard assignments can lead to less accurate parameter estimates and a less flexible model.

If $P(C_i = \hat{c}_i \mid X_i = x_i) \approx 1$ then we can correctly assume that the point belongs to that component. However, when $P(C_i = c_1 \mid X_i = x_i) \approx P(C_i = c_2 \mid X_i = x_i)$ we are making a crude approximation: both c_1 and c_2 might have been responsible for the generation of x_i. In general, the algorithm we discussed is not maximizing the likelihood of the observed samples x_i.

However, let's still consider hard-assignments. Let's also assume that we fix the covariance matrices of our GMM to $\Sigma_c = I$. We also fix the weights as $w_c = \frac{1}{K}$. In this case cluster assignment corresponds to the rule:

$$\hat{c}_i = \arg\max_c P(C_i = c | X_i = x_i) = \arg\min_c \|x_i - \mu_c\|^2$$

Our algorithm becomes:

- Compute the component or cluster whose centroid $\mu_{\hat{c}_i}$ is closest to our point and assign x_i to that cluster.
- Re-estimate the cluster centroids from the given points, and iterate until convergence.

This is the **K-Means clustering algorithm**. GMMs can also be applied to clustering tasks as a generalization of K-means. The algorithm we considered can be extended to handle soft assignments. We will see that a point is not completely associated to a single Gaussian component, but contributes to

the estimation of different components according to its cluster (component) posterior probability.

Consider the log-likelihood for our data:

$$\ell(\theta) = \sum_{i=1}^{N} \log f_{X_i}(x_i) = \sum_{i=1}^{N} \log \sum_{c=1}^{K} w_c \mathcal{N}(x_i;\, \mu_c,\, \Sigma_c)$$

Let's take the gradient with respect to μ_c:

$$\frac{\partial \ell}{\partial \mu_c} = \sum_i \frac{w_c \mathcal{N}(x_i;\, \mu_c,\, \Sigma_c)}{\sum_{c'} w_{c'} \mathcal{N}(x_i;\, \mu_{c'},\, \Sigma_{c'})} \Sigma_c^{-1}(x_i - \mu_c) = \sum_i \gamma_{c,i} \Sigma_c^{-1}(x_i - \mu_c)$$

which gives:

$$\hat{\mu}_c = \frac{\sum_i \gamma_{c,i} x_i}{\sum_i \gamma_{c,i}} \tag{3}$$

Notice that the responsibilities $\gamma_{c,i}$ depend on μ_c. If we knew the responsibilities we could compute μ_c as in (3). We can interpret (3) as a weighted empirical mean. The weight of each sample is the corresponding responsibility. The terms:

$$N_c = \sum_{i=1}^{N} \gamma_{c,i}, \quad F_c = \sum_{i=1}^{N} \gamma_{c,i} x_i$$

are also called zero and first order statistics. Note that we are summing over all samples in $\mathcal{D}$.

We can adopt a similar strategy for the covariance matrix, obtaining:

$$\hat{\Sigma}_c = \frac{1}{N_c} \sum_i \gamma_{c,i}(x_i - \hat{\mu}_c)(x_i - \hat{\mu}_c)^T = \frac{1}{N_c} \left(\sum_i \gamma_{c,i} x_i x_i^T \right) - \hat{\mu}_c \hat{\mu}_c^T$$

The terms:

$$S_c = \sum_i \gamma_{c,i} x_i x_i^T$$

are also called second order statistics. The weights can be re-estimated as:

$$\hat{w}_c = \frac{N_c}{N}, \quad \text{where } N = \sum_{c=1}^{K} N_c$$

Since we are not given $\gamma_{c,i}$, we can follow the same procedure we used for hard assignments: given θ, we estimate the responsibilities, ie the cluster or component posterior probabilities:

$$\gamma_{c,i} = P(C_i = c | X_i = x_i, \theta)$$

for each sample of our dataset $\mathcal{D}$, and given the responsibilities, we re-estimate the GMM parameters using the previous expressions. This procedure is a particular instance of an algorithm known as **Expectation-Maximization (EM)**.

Direct maximization of the GMM log-likelihood proved difficult (si è rilevata difficile) because of the form of the marginal log-density:

$$\log f_X(x) = \log \sum_{c=1}^{K} w_c \mathcal{N}(x;\, \mu_c,\, \Sigma_c)$$

On the contrary, we have seen that the optimization of joint cluster-feature likelihoods is straightforward: the joint likelihood consists of the product of cluster-conditional normal log-densities and cluster prior probabilities:

$$\log f_{X,C}(x, c) = \log \mathcal{N}(x;\, \mu_c,\, \Sigma_c) + \log w_c$$

and has a simple expression. The EM is an iterative procedure suited for the ML estimation of the parameters of complex likelihoods $f_X(x)$ that can be expressed through marginalization of joint likelihoods $f_{X,H}(x, h)$:

$$f_X(x) = \int f_{X,H}(x, h) dh = \int f_{X|H}(x|h) f_H(h) dh$$

Note that we do not make any assumption on what X represents. We simply assume that it's a random variable or a random vector, for which we observe a value x. H represents a latent (or hidden) random variable (or vector),

ie a random variable whose value has not been observed, ie is unknown. A latent random variable (or hidden random variable) is a random variable that cannot be directly observed in the data, but still influences the generative process or the model. For example, in Gaussian Mixture Models, the membership of a point to a cluster is a latent variable: we do not observe it, but it exists and affects how the data are generated. We will see that the EM transforms the maximization of a log-likelihood into a sequence of optimizations of expectations of the joint log-likelihood $\log f_{X,H}(x,h)$.

Let's consider again the marginal log-likelihood:

$$\log f_X(x) = \log \int f_{X,H}(x,h)dh$$

Given a density $Q(h)$ with the same support of $f_H(h)$, we can rewrite:

$$\log f_X(x) = \int Q(h) \log f_X(x)\, dh = \int Q(h) \log \frac{f_{X,H}(x,\, h)}{Q(h)}\, dh + \int Q(h) \log \frac{Q(h)}{f_{H|X}(h|x,\, \theta)}\, dh \tag{4}$$

[INTEGRAZIONE TEORICA]
Attenzione alla convenzione del segno: nelle slide il Prof. Cumani definisce $D_h(Q|f_{H|X}) = \mathbb{E}_Q[\log \frac{f_{H|X}}{Q}] \leq 0$, che è il negativo della KL divergenza standard $D_{KL}(Q|P) = \mathbb{E}_Q[\log \frac{Q}{P}] \geq 0$. Con questa convenzione: $\log f_X = \mathcal{L}_h + D_h$ dove $D_h \leq 0$, quindi $\mathcal{L}_h = \log f_X - D_h \geq \log f_X$. Ma nelle slide la conclusione è che $\mathcal{L}_h$ è un lower bound: questo è corretto nella convenzione standard dove $D_{KL}(Q|P) \geq 0$, ovvero $D_h(Q|f_{H|X}) = -D_{KL}(Q|f_{H|X}) \leq 0$, e quindi $\mathcal{L}_h = \log f_X - D_{KL}(Q|f_{H|X}) \leq \log f_X$. La decomposizione corretta è: $\log f_X(x;\theta) = \mathcal{L}_h(Q,\theta) + D_{KL}(Q|f_{H|X})$, e poiché $D_{KL} \geq 0$, si ha $\mathcal{L}_h \leq \log f_X$, ovvero $\mathcal{L}_h$ è un lower bound (ELBO).

thus $\mathcal{L}_h(Q(h),\, \theta)$ is a lower bound on the log-likelihood.

The EM algorithm optimizes the log-likelihood by iteratively:

- maximizing the lower bound $\mathcal{L}_h(Q(h),\, \theta)$ with respect to Q
- maximizing the lower bound $\mathcal{L}_h(Q(h),\, \theta)$ with respect to θ

From an initial set of parameters θ^0:

$$Q^0 = \arg\max_Q \mathcal{L}_h(Q(h),\, \theta^0)$$

$$\theta^1 = \arg\max_\theta \mathcal{L}_h(Q^0(h),\, \theta)$$

$$Q^1 = \arg\max_Q \mathcal{L}_h(Q(h),\, \theta^1)$$

$$\theta^2 = \arg\max_\theta \mathcal{L}_h(Q^1(h),\, \theta)$$

...

Let's consider the maximization of the lower bound with respect to Q, with $\theta = \theta^t$ fixed. We have shown that:

$$\mathcal{L}_h(Q,\, \theta^t) \leq \log f_X(x;\, \theta^t) \quad \text{and} \quad Q(h) = f_{H|X}(h|x;\, \theta^t) \implies \mathcal{L}_h(Q,\, \theta^t) = \log f_X(x;\, \theta^t)$$

therefore, we can maximize $\mathcal{L}_h(Q(h),\, \theta^t)$ with respect to Q by simply selecting:

$$Q^t(h) = f_{H|X}(h|x;\, \theta^t)$$

ie the posterior for H given $X = x$ and the fixed value θ^t for the parameters. Setting $Q^t(h) = f_{H|X}(h|x;\, \theta^t)$ does not change the log-likelihood $\log f_X(x;\, \theta^t)$, but reduces to zero the KL divergence:

$$\log f_X(x;\, \theta^t) = D_{KL}(Q^t(h) \,\|\, f_{H|X}(h|x;\, \theta^t))\big|_{=0} + \mathcal{L}_h(Q^t(h),\, \theta^t)$$

We can now maximize $\mathcal{L}_h(Q^t,\, \theta)$ w.r.t. θ. This requires computing:

$$\theta^{t+1} = \arg\max_\theta \mathbb{E}_{Q^t(h)}[\log f_{X,H}(x, h;\, \theta)]$$

Notice that the distribution we are taking the expectation with respect to does not involve θ anymore:

$$\mathbb{E}_{Q^t(h)}[\log f_{X,H}(x,h;\,\theta)] = \int Q^t(h)\log f_{X,H}(x,h;\,\theta)dh = \int f_{H|X}(h|x;\,\theta^t)\log f_{X,H}(x,h;\,\theta)dh$$

since the parameters used in the distribution $Q^t(h) = f_{H|X}(h|x;\,\theta^t)$ are fixed to θ^t (Q^t does not depend on θ, but on θ^t). We can also rewrite the problem as maximization of:

$$\mathbb{E}_{Q^t(h)}[\log f_{X,H}(x,h;\,\theta)] = \mathbb{E}_{Q^t(h)}[\log f_{X|H}(x|h;\,\theta)] + \mathbb{E}_{Q^t(h)}[\log f_H(h;\,\theta)]$$

which corresponds to the expression we used for the GMM. Since we are maximizing $\mathcal{L}_h(Q^t,\,\theta)$, we have $\mathcal{L}_h(Q^t,\,\theta^{t+1}) \geq \mathcal{L}_h(Q^t,\,\theta^t)$. $\mathcal{L}_h(Q^t,\,\theta^{t+1})$ is a lower bound of $\log f_X(x;\,\theta^{t+1})$, thus $\log f_X(x;\,\theta^{t+1}) \geq \mathcal{L}_h(Q^t,\,\theta^{t+1})$.

Maximization with respect to θ has increased both $\mathcal{L}_h$ and the KL-divergence, since, in general, $Q^t(h) \neq f_{H|X}(h|x;\,\theta^{t+1})$ unless we reach convergence. We thus have that $\log f_X(x;\,\theta^{t+1}) \geq \log f_X(x;\,\theta^t)$.

Maximization w.r.t. θ:

$$\log f_X(x;\,\theta^{t+1}) \geq \log f_X(x;\,\theta^t)$$

Indeed, the full chain of inequalities is:

$$\log f_X(x;\,\theta^t) = \mathcal{L}_h(Q^t,\,\theta^t) \leq \mathcal{L}_h(Q^t,\,\theta^{t+1}) \leq \log f_X(x;\,\theta^{t+1})$$

Indeed, maximization of $\mathcal{L}_h(Q^t,\,\theta)$ w.r.t. θ increases the log-pdf $\log f_X(x;\,\theta)$:

$$\log f_X(x;\,\theta^{t+1}) \geq \mathcal{L}_h(Q^t,\,\theta^{t+1}) \geq \mathcal{L}_h(Q^t,\,\theta^t) = \log f_X(x;\,\theta^t)$$

The algorithm iterates between 2 steps:

- **Expectation (E) step**: compute the posterior distribution $f_{H|X}(h|x;\,\theta^t)$ and compute the auxiliary function:

  $\mathcal{Q}(\theta,\,\theta^t) = \mathbb{E}_{f_{H|X}(h|x;\theta^t)}[\log f_{X,H}(x,h;\,\theta)]$

- **Maximization (M) step**: maximize $\mathcal{Q}(\theta,\,\theta^t)$ w.r.t. θ to obtain θ^{t+1}:

$$\theta^{t+1} = \arg\max_\theta \mathcal{Q}(\theta, \theta^t)$$

Under very weak conditions it can be shown that the EM algorithm converges to a saddle point (punto di sella) of the log-likelihood $\ell(\theta)$. Sufficient conditions for θ to be a local maximum exist, but are not easy to verify in practice. The saddle point will depend on the initial set of parameters. The choice of a good starting point is important for the estimation of good models. It is sometimes useful to apply the EM algorithm several times with different starting points.

Let's apply the algorithm to the GMM. We have a set of N hidden variables $h = \{C_1, \ldots, C_N\}$ that represent the cluster assignments, ie the assignment of each sample to a component of the GMM. The GMM specifies the joint likelihood for samples and cluster assignments. For a single sample:

$$f_{X_i,C_i}(x_i, c) = w_c \mathcal{N}(x_i;\, \mu_c,\, \Sigma_c)$$

The cluster random variable C_i has a Categorical prior distribution:

$$P(C_i = c) = w_c$$

whereas the sample conditional likelihood is:

$$f_{X_i|C_i}(x_i|c) = \mathcal{N}(x_i;\, \mu_c,\, \Sigma_c)$$

We assume that samples are independent given the model parameters, so that we can express the log-likelihood for all the training set samples as:

$$\log f_{X_1,\ldots,X_N,C_1,\ldots,C_N}(x_1, \ldots, x_N, c_1, \ldots, c_N;\, \theta) = \sum_{i=1}^{N} \log f_{X_i,C_i}(x_i, c_i;\, \theta)$$

The EM algorithm requires computing the posterior for the hidden variables $C_1, \ldots, C_N \mid X_1 = x_1, \ldots, X_N = x_N, \theta$. Due to the independence assumptions, also the posterior distribution factorizes as:

$$f_{C_1,\ldots,C_N|X_1,\ldots,X_N}(c_1, \ldots, c_N|x_1, \ldots, x_N;\, \theta) = \prod_{i=1}^{N} P(C_i = c_i|X_i = x_i;\, \theta)$$

The E-step requires computing the auxiliary function:

$$\mathcal{Q}(\theta,\, \theta^t) = \mathbb{E}_{C_1,\ldots,C_N|X_1=x_1,\ldots,X_N=x_N;\,\theta^t}[\log f_{X_1,\ldots,X_N,C_1,\ldots,C_N}(x_1,\ldots,x_N,c_1,\ldots,c_N;\,\theta)]$$

$$= \sum_{i=1}^{N} \mathbb{E}_{C_i|X_i=x_i;\,\theta^t}[\log f_{X_i,C_i}(x_i,c_i;\,\theta)]$$

The EM algorithm becomes:

- **E-step**: compute $\gamma_{c,i} = P(C_i = c|X_i = x_i;\,\theta^t)$. The auxiliary function is:

  $\mathcal{Q}(\theta,\, \theta^t) = \sum_{i=1}^{N}\sum_{c=1}^{K} \gamma_{c,i}[\log \mathcal{N}(x_i;\,\mu_c,\,\Sigma_c) + \log w_c]$

- **M-step**: maximize $\mathcal{Q}(\theta,\, \theta^t)$ w.r.t. $\mathcal{M}, \mathcal{S}, \mathbf{w}$, subject to $\sum_{k=1}^{K} w_k = 1$:

  $\mu_c^* = \frac{\sum_i \gamma_{c,i} x_i}{\sum_i \gamma_{c,i}}$

  $\Sigma_c^* = \frac{\sum_i \gamma_{c,i}(x_i-\mu_c^*)(x_i-\mu_c^*)^T}{\sum_i \gamma_{c,i}}$

  $w_c^* = \frac{\sum_i \gamma_{c,i}}{\sum_{i,c'} \gamma_{c',i}}$

The new estimate of the parameters is $\theta^{t+1} = (\mathcal{M}^{t+1}, \mathcal{S}^{t+1}, \mathbf{w}^{t+1})$:

$$\mathcal{M}^{t+1} = \{\mu_1^*,\ldots,\mu_K^*\}, \quad \mathcal{S}^{t+1} = \{\Sigma_1^*,\ldots,\Sigma_K^*\}, \quad \mathbf{w}^{t+1} = \{w_1^*,\ldots,w_K^*\}$$

Just as we used Gaussian densities for modeling the samples of different classes in a classification task, we can use GMM to model the class conditional distribution. We can, for example, assume that the samples of class c are generated by a GMM with parameters $(\mathcal{M}_c, \mathcal{S}_c, \mathbf{w}_c)$. For each class we want to recognize, we can compute the ML estimate of a GMM for the samples of that class. We can then use the estimated densities to compute class conditional log-likelihoods and class posterior distributions or log-likelihood ratios.

The MVG for classification fits a Gaussian density to samples of each class:

$$X_i|C_i = c \sim \mathcal{N}(\mu_c, \Sigma_c)$$

$$f_{X_t|C_t}(x_t|c) = \mathcal{N}(x_t;\, \mu_c,\, \Sigma_c)$$

$$P(C_t = c|X_t = x_t) \propto \mathcal{N}(x_t;\, \mu_c,\, \Sigma_c)P(C_t = c)$$

The GMM for classification chooses the number of components K_c for each class and fits a GMM with K_c components to samples of each class:

$$X_i|C_i = c \sim GMM(\mathcal{M}_c, \mathcal{S}_c, \mathbf{w}_c)$$

$$f_{X_t|C_t}(x_t|c) = \sum_{k=1}^{K_c} w_{c,k}\mathcal{N}(x_t;\, \mu_{c,k},\, \Sigma_{c,k})$$

$$P(C_t = c|X_t = x_t) \propto P(C_t = c) \cdot \sum_{k=1}^{K_c} w_{c,k}\mathcal{N}(x_t;\, \mu_{c,k},\, \Sigma_{c,k})$$

We can also exploit GMMs clustering capabilities for open-set multiclass classification. The difficulty in open-set classification consists in building a robust model for the none-of-the-others class. This class is usually very heterogeneous, and explicit modeling of its sub-components requires labeled examples of its possible objects. We can partially alleviate the issue using a GMM. We can assume that samples of known classes can be modeled by MVG distributions. We can collect a large set of unlabeled samples for the none-of-the-others class. We can model the samples of the none-of-the-others class using a GMM. The GMM will find homogeneous clusters (sub-classes) of the none-of-the-others population, and can be used as an estimate for the conditional density of a test sample assuming that it belongs to the none-of-the-others class. Given the complexity of the task, and due to the fact that different amounts of parameters are used for modeling the none-of-the-others class, these kinds of models often provide class-conditional likelihoods that are not calibrated, and may require further score processing to obtain good decisions.

As we did for MVG, we can train GMM with diagonal covariance matrices to reduce the numbers of parameters to estimate (reducing overfitting and

computational costs). The solution is again given by the diagonals of Σ_c that we defined before. We may need more components to model more complex distributions. The diagonal covariance assumption does not correspond to the Naive Bayes assumption in this case! The Naive Bayes assumption would correspond to training a different GMM (possibly with different number of components) for each subset of features that is assumed independent from the other features.

We can also assume that all components of a GMM have the same covariance matrix (tied GMM). Note that in this case we are tying the components of a single GMM, ie the Gaussian components of the GMM of a single class. This is different from the Tied Gaussian Model, where parameters were shared across classes.

Initialization plays an important role in GMM training. K-means can be used as initializer. Hard-assignment with isotropic covariances $\rightarrow$ K-means. An alternative approach is the LBG. Initialization is important in GMM training, and K-means can be used as an initializer. In fact, if we use hard assignments and assume isotropic covariances for all components, the procedure becomes equivalent to the K-means algorithm. Thus, K-means can be seen as a special case of GMM training under these assumptions.

The LBG algorithm:

- GMM: split the components of a G-components GMM
- $\mu_c^{\pm} = \mu_c \pm \varepsilon$
- to obtain an initial $2G$-components GMM
- run the EM algorithm until convergence for the $2G$-components GMM
- iterate until the desired number of Gaussians is reached

A good value for ε can be a displacement along the principal eigenvector of the covariance matrix Σ_c.

The problem is: what's the right number of Gaussians? If we increase the number of components, the likelihood will increase. We cannot choose based on likelihood alone. We can resort to cross-validation. We also need to pay attention to degenerate models. As we said at the beginning, the log-likelihood for a GMM, as long as we have at least two components, is unbounded. The EM algorithm will usually find local maximums that are well-behaved, however, especially if we have too many components, we may obtain degenerate

models which cause numerical issues. Some heuristics can be used to force models to be well-behaved. We can also modify our initialization so that the algorithm may end up in a different local maximum.

9 Score Calibration and Fusion

Score Calibration and Fusion

Score calibration and fusion are techniques used to improve the reliability and effectiveness of classification systems, especially when combining outputs from multiple models or sources. **Score calibration** refers to the process of transforming the raw scores produced by a classifier into well-calibrated probabilities or likelihood ratios, so that they accurately reflect the true likelihood of each class. **Score fusion** involves combining the scores from different classifiers or systems to make a more robust and accurate final decision. These methods are essential in applications where decisions must be based on multiple sources of information or where the interpretation of scores as probabilities is important for downstream tasks.

In many cases, the scores output by classifiers do not have a clear probabilistic interpretation and do not represent log-likelihood ratios for the application evaluation population. For example, SVM scores cannot be directly interpreted as probabilities. In regularized logistic regression, strong regularization can distort the scores so that they no longer accurately reflect class probabilities. Even generative models can produce poorly calibrated scores if they suffer from overfitting or if there is a mismatch between the training and test data distributions. As a result, making decisions directly based on classifier scores can lead to miscalibration.

For binary problems we have seen that decision rules take the form of a comparison of the classifier score with a threshold. To reduce miscalibration we can adopt two strategies. We can use a validation set to find a (close-to) optimal threshold for a given application. Note that we need to know the target application when estimating the threshold. Alternatively, we can look for functions that transform the classifier scores into approximately well-calibrated LLRs, so that optimal decisions can be obtained for different applications. Note that it is application independent and that prior knowledge of the application can still help for some models, but a rough estimate of possible applications is often sufficient.

Let's consider the second approach. We want to compute a transformation function f that maps a classifier score s into a well-calibrated score $s_{cal} = f(s)$. We consider monotone functions f, as to preserve the fact that higher non-calibrated scores should favor class H_T and lower non-calibrated scores should favor class H_F.

There are several methods to find a monotonic calibration function that transforms classifier scores into well-calibrated scores. **Isotonic regression** is a flexible, non-parametric approach that fits a monotonic function to the data. **Prior-weighted logistic regression** assumes an affine (linear) calibration function and incorporates class priors, making it simple and effective for many applications. **Generative score models**, on the other hand, assume a specific probabilistic model for the distribution of scores in each class and estimate the calibration function under these constraints. These methods help ensure that the calibrated scores can be interpreted as reliable probabilities or likelihood ratios.

Isotonic regression is a non-linear, monotonic transformation that provides optimal calibration for the data it is trained on. The resulting function is piecewise non-linear, which means it is made up of several linear or constant segments joined together. This approach may require interpolation for scores that were not seen during training, and it does not allow extrapolation outside the range of training scores. Additionally, isotonic regression can be computationally expensive to evaluate when the calibration training set is large.

Score models require assumptions about the calibration transformation, such as using affine (linear) mapping, or about the distribution of class scores. These models are estimated over a training set and allow for extrapolation outside the range of training scores. They are typically fast to evaluate, but may provide a good fit only for a limited range of operating points. An example of a score model is prior-weighted logistic regression. In the prior-weighted logistic regression, the non-calibrated scores are treated as 1-D feature vectors and an affine mapping is assumed from the non-calibrated scores to the calibrated scores:

$$f(s) = \alpha s + \gamma$$

Since $f(s)$ should produce well-calibrated LLRs, $f(s)$ can be interpreted as the log-likelihood ratio for the two class hypotheses:

$$f(s) = \log \frac{f_{S|C}(s|H_T)}{f_{S|C}(s|H_F)} = \alpha s + \gamma$$

The class posterior probabilities for prior $\tilde{\pi}$ correspond to:

$$\log \frac{P(C = H_T|s)}{P(C = H_F|s)} = \alpha s + \gamma + \log \frac{\tilde{\pi}}{1 - \tilde{\pi}} = \alpha s + \beta$$

We can employ the (non-regularized) prior-weighted logistic regression model to learn the model parameters α, β from a set of training scores (we will refer to the set of samples employed to estimate the calibration parameters as calibration training set in the following). In practice, we are treating scores as if they were 1-D feature vectors. As for model training and validation set, also the calibration set should be an independent dataset that does not overlap with neither the model training nor the validation set. The calibration transformation corresponds to the transformation of a prior-weighted logistic regression model. Once we have estimated α and β, the calibrated score $f(s)$ can be computed as:

$$f(s) = \alpha s + \gamma = \alpha s + \beta - \log \frac{\tilde{\pi}}{1 - \tilde{\pi}}$$

Note that we have to specify a prior $\tilde{\pi}$, and we are still effectively optimizing the calibration for a specific application $\tilde{\pi}$. However, often this approach will provide good calibration for a wider range of different applications similar to the target one. We can also modify the prior-weighted logistic regression objective to compute directly α and γ:

$$R(\alpha, \gamma) = \sum_i w_i \log \left(1 + e^{-z_i(\alpha s + \gamma + \log \frac{\tilde{\pi}}{1-\tilde{\pi}})}\right), \quad w_i = \begin{cases} \tilde{\pi}/n_T & \text{if } z_i = +1 \\ (1 - \tilde{\pi})/n_F & \text{if } z_i = -1 \end{cases}$$

Alternatively to logistic regression, we can employ generative models to estimate calibrated LLRs. We model the class-conditional score distribution for the two classes $S \mid H_T$ and $S \mid H_F$, and the calibration transformation is given by the LLR for the score model:

$$f_{cal}(s) = \log \frac{f_{S|H_T}(s)}{f_{S|H_F}(s)}$$

For example, we can employ a Gaussian model to estimate the two densities:

$$f_{S|H_T}(s) = \mathcal{N}(s|\mu_T, v_T), \quad f_{S|H_F}(s) = \mathcal{N}(s|\mu_F, v_F)$$

Tied models $v_T = v_F = v$ are usually employed, as they result in monotonic transformations.

In all cases, we need a calibration set to estimate the transformation. The calibration set must differ from the validation set and the evaluation sets, otherwise we would get biased results. Typically, there are two scenarios. In the first scenario, miscalibration is caused by non-probabilistic scores, or by overfitting or underfitting models, but evaluation and training populations are similar. In this case, the calibration set can be extracted from the training set material. Calibration in this setting allows us to recover a probabilistic interpretation of the scores. Since the calibration model is trained on 1-D data, the risk of overfitting is drastically reduced and regularization term is usually not needed in the calibration objective. In the second scenario, miscalibration is due to a mismatch between the training population and the application or evaluation population. Here, the calibration set (and, if we want to assess calibration quality, the calibration validation set as well) should mimic the application population. It is important to collect data that matches the characteristics of the application scenario. However, the amount of required matching data is usually smaller than what is needed to train a full classifier. Some models, eg Gaussian models, can be extended to exploit unlabeled samples through unsupervised or semi-supervised training, which are less expensive to acquire.

The approach we followed up to now would require that we split the training material in 3 parts:

- Model training set
- Calibration training set (samples that are scored with the trained classifier and are used to compute calibration parameters)
- (Calibration) validation set (samples that are scored with the trained model and whose scores are calibrated with the calibration model, used to evaluate the performance of the complete system, including the calibration model)

Calibration training set: Sono campioni che vengono valutati (scored) dal classificatore già addestrato. I punteggi ottenuti vengono usati per stimare i parametri del modello di calibrazione (ad esempio, per la regressione logistica di calibrazione).

Calibration validation set: Sono campioni che vengono anch'essi valutati dal classificatore e poi calibrati dal modello di calibrazione. Questo

set serve per valutare le prestazioni finali del sistema completo (classificatore + calibrazione), senza rischio di overfitting sui dati usati per stimare la calibrazione.

Splits can be performed in several ways, for simplicity, we assume that our calibration training and validation sets are extracted from our former validation set. This allows us to re-use models we already trained.

Configurazione corrente (senza dataset di calibrazione):

```
Training set:  [ Model train | Validation ]
```

Train – Calibration – Validation (split a 3 parti):

```
Training set:  [ Model train | Calibration train | Validation ]
```

Calibration e validation estratti dal former validation set:

```
Training set:  [ Model train |------- Former validation --------|]
[ Calibration train | (Cal.)  Validation ]
```

An issue is that the calibration and validation sets become both smaller. If we enlarge the calibration set, our evaluation metrics computed over the validation set become less reliable. If we reduce the calibration set, our calibration model becomes less reliable. In general, all three partitions would benefit from more data. A possible approach to address the limited amount of data is to resort to K-fold cross-validation. Single-fold partitions the dataset in separate sets. K-fold cross-validation extends this approach by repeatedly splitting the data into model training, calibration training, and calibration validation sets in K different ways. This ensures that every sample is used at least once for each role—training the classifier, training the calibration model, and evaluating the complete system—making the most efficient use of all available data.

We start considering a simple set-up, where we need two partitions: these could be model training and validation sets extracted from a training dataset, but also could be the calibration and actual validation sets extracted from our former validation set. In both cases, we would like to use the data to train some model and to evaluate its performance. K-fold cross-validation splits the dataset into K partitions, usually with similar numbers of samples and preserving the original class ratios. The model is trained iteratively using K-1 folds, while the remaining fold is used for scoring. After all iterations, the scores from each fold are pooled together, and the desired metric is evaluated on this combined set. This metric is then used for model selection

or hyperparameter tuning. It is important that all K models are trained with the same setup, including identical preprocessing steps and hyperparameter values. For some metrics, it is essential to compute the metric over the pooled scores from all folds, rather than averaging the metric computed separately on each fold. Averaging per-fold metrics can lead to biased results and should be avoided. Additionally, the same threshold must be used for all scores when evaluating the metric.

Once we have selected the approach (model selection) and tuned the model parameters, we still need to choose which model to use in practice. We have trained K models, all of them using a subset of the data. We cannot choose any of them based on their performance on the corresponding validation fold. Fold data are different, and the comparison would then make little sense. Moreover, in many cases increasing the training set size is beneficial, but each model was trained using only approximately a fraction $\frac{K-1}{K}$ of the training set. The solution is to train one more model over the whole training set, using the method we selected and the hyperparameters we estimated in the previous step. The model M will then be used to compute the scores for the application data. Note that the model selection and hyperparameter tuning has been performed using models that are different from M. The selected values may not generalize well for the final model. Furthermore, each model M_i may be affected by different levels of miscalibration. Ideally, to minimize these issues we want each model M_i to be as similar as possible to the final model M.

Leave-one-out (LOO) cross validation is a special case of K-fold cross-validation where K is set to the total number of training samples N. In this approach, each time a single sample is left out from the training set, and the model is trained on all the remaining $N-1$ samples. The left-out sample is then scored using the trained model. This process is repeated for every sample, resulting in a pooled validation set of N samples, each evaluated by a model trained on nearly all the data. While the models obtained in each iteration are usually very similar, LOO requires training as many models as there are samples, which can be computationally expensive for large datasets. We choose K as large as we can afford. Large values of K lead to more robust analysis and more time required. Small values of K lead to less robust analysis and less time required.

We can extend the approach to deal with our three-set partitioning using a two-step approach. The first step is to apply K-fold to train the classifier:

- Train K classifiers, each without fold k (model R_k)
- Score each fold k with model R_k
- Train a classifier R_F over the whole training set
- Pool the scores of each fold to obtain a score set, that we will use as calibration set

The second step is to apply K-fold over the calibration scores, using a different randomized shuffle:

- Train K calibration models C_k
- Train a calibration model over all pooled scores C_F
- Calibrate the scores of each fold k with model C_k
- Pool the calibrated scores and evaluate the model performance over the pooled scores to choose the best configuration
- Choose the classification system according to pooled scores results. Classify the application samples: apply classification model R_F for the chosen system followed by its calibration model C_F.

In many cases, we may have competing models that achieve similar performance. Sometimes, even when their overall performance differs, different classifiers can extract different information from the feature vectors. This raises the question: can we combine these models to further improve classification performance for a given test sample? A simple approach is to use a majority-voting scheme. For each sample to be classified, we compute the prediction of each classifier under consideration and assign the label that is selected most frequently by the models. While this method can be effective, it has some limitations: it requires tie-breaking rules in case of a draw, and it does not take into account the confidence or strength with which each classifier supports its labeling hypothesis.

A tie-breaking rule is a procedure used to decide the final label when two or more classes receive the same number of votes from the classifiers. For example, one might randomly select among the tied classes or always choose a specific class in case of a tie.

Consider a binary task with well calibrated scores and three classifiers that provide, for a sample x, the scores:

$$s_1 = 10.0, \quad s_2 = -0.1, \quad s_3 = -0.1$$

If our threshold is 0, then majority voting would label the sample as class H_F. The scores s_1, s_2, s_3 tell us that two systems are very uncertain, with a slight bias towards class H_F, but the first one is very confident in favor of class H_T.

If our models provide scores as output, then we can try combining the scores for a given sample. For example, we may try computing the average or the sum of the scores. If our models provide LLRs, and the class-conditional likelihoods provided by the systems were independent, then the sum would be correct. If our models are strongly correlated the sum becomes incorrect, as some systems are providing only partial or even no additional information. We can consider weighting the contribution of the different system terms. Given scores $s_1 \dots s_m$ for a test sample x, we can compute a fused score as:

$$s_{fused} = \alpha_1 s_1 + \alpha_2 s_2 + \alpha_3 s_3 + \cdots + \alpha_m s_m + \gamma$$

We have the problem of estimating the weights α_i and the bias term γ. If we stack the scores and the weights as:

$$s = \begin{pmatrix} s_1 \\ \vdots \\ s_m \end{pmatrix}, \quad \alpha = \begin{pmatrix} \alpha_1 \\ \vdots \\ \alpha_m \end{pmatrix}$$

the fused score can be represented as:

$$s_{fused} = \alpha^T s + \gamma$$

The output score s_{fused} is obtained as an affine transformation of the score "feature" vector $\mathbf{s}$. We can then employ a method like logistic regression to estimate the weights $\boldsymbol{\alpha}$ and the bias term γ (if we want a LLR-like fused scores we need to compensate the logistic regression model parameters for the training prior log-odds as in the calibration case to obtain γ). This is similar to calibration and typically provides calibrated scores as output, but rather than 1-D samples we now have m-dimensional vectors, containing the scores of m classifiers for each sample. We can exploit the calibration partition of our set to estimate the score fusion weights. If we had a single system, we would obtain exactly the calibration procedure we analyzed earlier.

For multiclass problems the optimal decisions cannot be represented anymore as a comparison of a score with a threshold. In this case, we have seen that it's more complex to separate the contributions to the cost due to poor discrimination and poor calibration. Calibration methods can be extended to calibrate also scores of multiclass recognizers, so that they can be interpreted as class-conditional log-likelihoods that can then be combined with application priors and costs to obtain optimal decisions. Typically, multiclass logistic regression models are employed to estimate calibration or fusion weights and biases for the different classes and classifiers.